

Design System Reference & AI Guide

Every component, template, hook, provider, token, theme and utility — with rendered examples of every variant in light, dark and Arabic RTL, plus ready-to-use prompts for AI assistants.

36

components & templates

3

themes: light · dark · black

LTR/RTL

English & Arabic

17

AI prompts

variant="primary"

Continue

small

Continue

medium

Continue

large

✓ Next >

leading + trailing

⌂ Saving

loading

Disabled

disabled

variant="secondary"

Continue

small

Continue

medium

Continue

large

✓ Next >

leading + trailing

⌂ Saving

loading

Disabled

disabled

variant="outline"

Continue

small

Continue

medium

Continue

large

✓ Next >

leading + trailing

⌂ Saving

loading

Disabled

disabled

variant="ghost"

Continue

small

Continue

medium

Continue

large

✓ Next >

leading + trailing

⌂ Saving

loading

Disabled

disabled

variant="danger"

Continue

small

Continue

medium

Continue

large

✓ Next >

leading + trailing

⌂ Saving

loading

Disabled

disabled

variant="link"

[Continue](#)

small

[Continue](#)

medium

[Continue](#)

large

✓ Next >

leading + trailing

⌂ Saving

loading

[Disabled](#)

disabled

Generated October 2026 from the source in [src/design-system](#). Screenshots are real renders of the components (react-native-web, Chromium). Native-only pieces — the platform date grid — are shown with a web stand-in.

Contents

1. [Getting started](#)
2. [Architecture & the RNTheme boundary](#)
3. [Foundations: tokens & themes](#)
4. [Providers, hooks & utilities](#)
5. Components
 - **Typography & icons:** [Text](#), [Heading](#), [Icon](#)
 - **Actions:** [Button](#), [IconButton](#), [Link](#)
 - **Form controls:** [Input](#), [PasswordInput](#), [SearchInput](#), [TextArea](#), [FormField](#), [Checkbox](#), [Radio](#), [Switch](#), [Select](#), [DatePicker](#)
 - **Data display:** [Badge](#), [Chip](#), [Avatar](#), [Divider](#), [Card](#), [ListItem](#), [List](#)
 - **Feedback & status:** [Alert](#), [Toast](#), [Spinner](#), [Skeleton](#), [EmptyState](#), [ErrorState](#)
 - **Disclosure & navigation:** [Accordion](#), [Tabs](#), [Tooltip](#)
 - **Overlays:** [Modal](#), [BottomSheet](#)
 - **Templates:** [NumPad](#), [OTPTemplate](#)
6. [AI prompts](#)
7. [Appendix: type aliases & complete export list](#)

1 • Getting started

Install

```
# inside the design-system repo: build a tarball (npm i && npm pack; prepack rebuilds dist/)
npm run pack
```

```
# in your app
npm i ../Bunyan-Design/bunyan-design-system-0.0.1.tgz
npm i react-native-svg @react-native-community/datetimetypepicker
```

Peer dependencies: `react ≥ 18.2`, `react-native ≥ 0.74`, `react-native-svg ≥ 15`, `@react-native-community/datetimetypepicker ≥ 8`. Metro resolves the `"react-native": "src/index.ts"` entry (TypeScript source); other bundlers use `dist/`.

Wrap your app

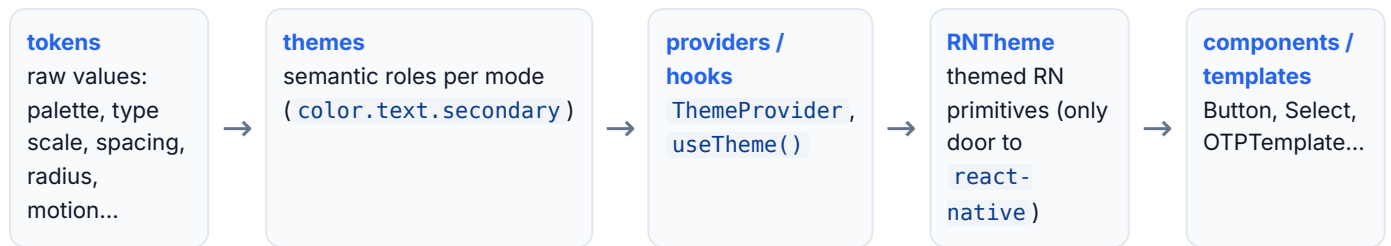
```
import { ThemeProvider, ToastProvider } from '@bunyan/design-system';

export default function App() {
  return (
    <ThemeProvider initialPreference="system" locale="ar-SA" blackForSystemDark>
      <ToastProvider maxVisible={3}>
        <RootNavigator />
      </ToastProvider>
    </ThemeProvider>
  );
}
```

Scripts

<code>npm run pack</code>	<code>npm i && npm pack</code> — installs dependencies, then <code>prepack</code> cleans and rebuilds <code>dist/</code> and writes <code>bunyan-design-system-<version>.tgz</code> .
<code>npm run verify</code>	Typecheck + Jest (includes the test that fails if any file other than <code>RNTheme/native.ts</code> imports <code>react-native</code>).
<code>npm run build / clean</code>	Compile <code>src</code> to <code>dist</code> / delete <code>dist</code> .
<code>npm test, test:watch, test:coverage</code>	Jest with <code>@testing-library/react-native</code> .
<code>npm run storybook[:ios :android :web]</code>	On-device Storybook via the Expo app in <code>showcase/</code> .

2 · Architecture & the RNTheme boundary



Dependencies only flow left to right. Components never import product code, and design-system code never imports `react-native` directly: `components/RNTheme/native.ts` is the single boundary file and a test enforces it.

```
// inside a component
import { Pressable, View, type ViewRef } from '../RNTheme';

// in app code
import { RNTheme, createThemedComponent } from '@bunyan/design-system';
<RNTheme.View themeStyle={({ theme }) => ({ padding: theme.spacing.lg })} />

const Surface = createThemedComponent(RNTheme.View, {
  displayName: 'Surface',
  baseStyle: ({ theme }) => ({ backgroundColor: theme.color.surface.primary, borderRadius:
theme.radius.lg }),
});
```

Themed primitives

PRIMITIVE	THEME DEFAULTS
<code>View</code> , <code>Pressable</code> , <code>Image</code> , <code>RNModal</code>	Layout-neutral; add <code>themeStyle</code> and ref forwarding.
<code>RNText</code>	Text colour, direction-aware font family, <code>writingDirection</code> , font scaling capped at 2×.
<code>TextInput</code>	Text, placeholder, selection and cursor colours; font family; font scaling.
<code>ScrollView</code>	Indicator style per mode; <code>keyboardShouldPersistTaps="handled"</code> .
<code>ActivityIndicator</code>	Primary colour.
<code>RNSwitch</code>	Track colours.
<code>KeyboardAvoidingView</code>	<code>behavior="padding"</code> on iOS.
APIs	<code>Animated</code> , <code>Platform</code> , <code>Linking</code> , <code>StyleSheet</code> , <code>I18nManager</code> , <code>AccessibilityInfo</code> , <code>Appearance</code> ; types <code>ViewStyle</code> , <code>TextStyle</code> , <code>PressableProps</code> ...; refs <code>ViewRef</code> , <code>TextRef</code> , <code>TextInputRef</code> , <code>ScrollViewRef</code> .

createThemedComponent options

PROP	TYPE	DEFAULT
<code>displayName</code> required	<code>string</code> Shown in React DevTools and error messages.	—
<code>defaultProps</code>	<code>(context: ThemeContextValue) => Partial<P></code> Theme-aware props applied first; anything passed explicitly wins.	—

PROP	TYPE	DEFAULT
<code>baseStyle</code>	<code>(context: ThemeContextValue) => ThemedStaticStyle<P></code> Theme-aware base style; merged underneath the caller's <code>style</code> .	—

Style layers merge as `baseStyle` → `themeStyle` → `style` (explicit `style` wins). Pressable-style `(state) => style` functions keep working, and an explicit `undefined` prop never erases a theme default.

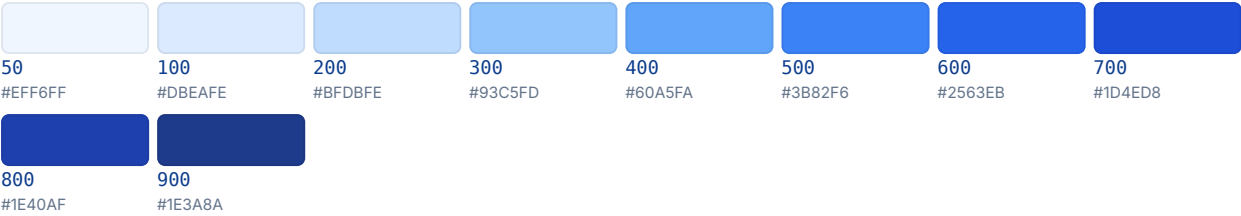
RTL model

Direction is provider-scoped: `locale="ar-SA"` (or `direction="rtl"`) makes a subtree RTL without touching the global `I18nManager`. React Native already mirrors `row` and swaps `left` / `right` when the app runs natively in RTL, so `logicalRow`, `logicalText` and `logicalAlignItems` only mirror when the requested direction differs from the native one. Always use them instead of hard-coding `row-reverse`.

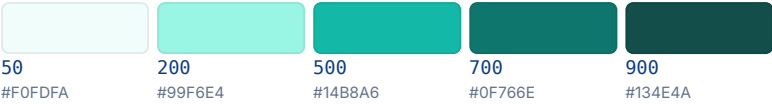
3 · Foundations: tokens & themes

Palette (`tokens.palette`)

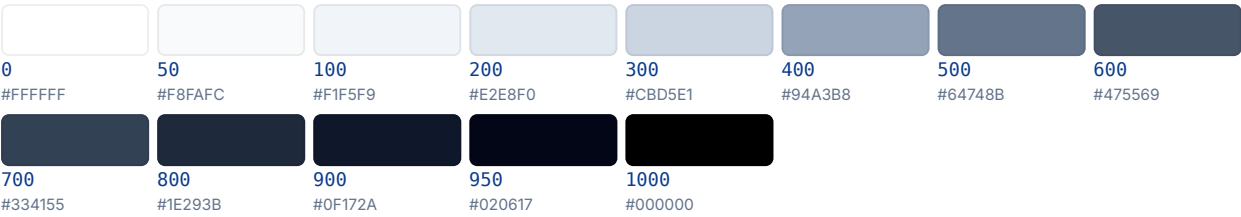
blue



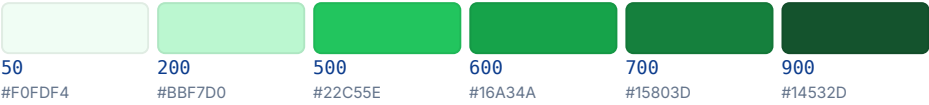
teal



gray



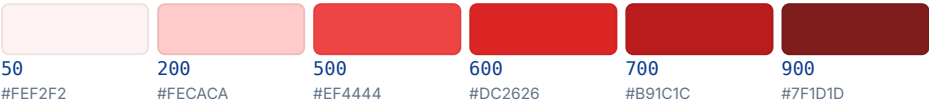
green



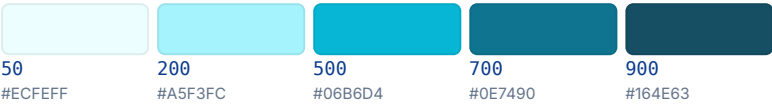
amber



red



cyan



Semantic colours (`theme.color`)

TOKEN	LIGHT	DARK	BLACK
<code>color.primary.default</code>	#2563EB	#60A5FA	#60A5FA
<code>color.primary.hover</code>	#1D4ED8	#93C5FD	#93C5FD
<code>color.primary.pressed</code>	#1E40AF	#BFD8FE	#BFD8FE
<code>color.primary.subtle</code>	#EFF6FF	#1E3A8A	#1E3A8A
<code>color.primary.contrast</code>	#FFFFFF	#020617	#020617
<code>color.secondary.default</code>	#0F766E	#14B8A6	#14B8A6
<code>color.secondary.hover</code>	#134E4A	#99F6E4	#99F6E4
<code>color.secondary.pressed</code>	#134E4A	#99F6E4	#99F6E4

TOKEN	LIGHT	DARK	BLACK
color.secondary.subtle	 #F0FDF4	 #134E4A	 #134E4A
color.secondary.contrast	 #FFFFFF	 #020617	 #020617
color.neutral.default	 #475569	 #CBD5E1	 #CBD5E1
color.neutral.subtle	 #F1F5F9	 #1E293B	 #1E293B
color.neutral.strong	 #0F172A	 #F8FAFC	 #F8FAFC
color.neutral.contrast	 #FFFFFF	 #020617	 #020617
color.success.default	 #16A34A	 #22C55E	 #22C55E
color.success.subtle	 #F0FDF4	 #14532D	 #14532D
color.success.border	 #BBF7D0	 #15803D	 #15803D
color.success.text	 #14532D	 #BBF7D0	 #BBF7D0
color.warning.default	 #D97706	 #F59E0B	 #F59E0B
color.warning.subtle	 #FFFBEF	 #78350F	 #78350F
color.warning.border	 #FDE68A	 #D97706	 #D97706
color.warning.text	 #78350F	 #FDE68A	 #FDE68A
color.error.default	 #DC2626	 #EF4444	 #EF4444
color.error.subtle	 #FEF2F2	 #7F1D1D	 #7F1D1D
color.error.border	 #FECACA	 #B91C1C	 #B91C1C
color.error.text	 #7F1D1D	 #FECACA	 #FECACA
color.information.default	 #0E7490	 #06B6D4	 #06B6D4
color.information.subtle	 #ECFEFF	 #164E63	 #164E63
color.information.border	 #A5F3FC	 #0E7490	 #0E7490
color.information.text	 #164E63	 #A5F3FC	 #A5F3FC
color.background.primary	 #FFFFFF	 #0F172A	 #000000
color.background.secondary	 #F8FAFC	 #020617	 #020617
color.background.inverse	 #020617	 #FFFFFF	 #FFFFFF
color.surface.primary	 #FFFFFF	 #1E293B	 #020617
color.surface.secondary	 #F8FAFC	 #0F172A	 #000000
color.surface.elevated	 #FFFFFF	 #334155	 #0F172A
color.surface.inverse	 #0F172A	 #FFFFFF	 #FFFFFF
color.border.primary	 #CBD5E1	 #475569	 #334155
color.border.secondary	 #E2E8F0	 #334155	 #1E293B
color.border.focus	 #2563EB	 #60A5FA	 #60A5FA
color.border.error	 #DC2626	 #EF4444	 #EF4444
color.border.success	 #16A34A	 #22C55E	 #22C55E
color.text.primary	 #0F172A	 #F8FAFC	 #F8FAFC
color.text.secondary	 #475569	 #CBD5E1	 #CBD5E1
color.text.tertiary	 #64748B	 #94A3B8	 #94A3B8
color.text.inverse	 #FFFFFF	 #020617	 #020617
color.text.link	 #1D4ED8	 #93C5FD	 #93C5FD
color.disabled.background	 #F1F5F9	 #1E293B	 #1E293B
color.disabled.border	 #E2E8F0	 #334155	 #334155
color.disabled.text	 #94A3B8	 #64748B	 #64748B
color.overlay.scrim	 rgba(2, 6, 23, 0.64)	 rgba(0, 0, 0, 0.72)	 rgba(0, 0, 0, 0.72)

TOKEN	LIGHT	DARK	BLACK
color.overlay.subtle	☒ rgba(2, 6, 23, 0.12)	☐ rgba(255, 255, 255, 0.12)	☐ rgba(255, 255, 255, 0.12)
color.overlay.transparent	☐ transparent	☐ transparent	☐ transparent

Typography (`tokens.typography`)

xs	12px / 16px	Bunyan بنيان
sm	14px / 20px	Bunyan بنيان
md	16px / 24px	Bunyan بنيان
lg	18px / 28px	Bunyan بنيان
xl	20px / 30px	Bunyan بنيان
xxl	24px / 34px	Bunyan بنيان
displaySm	30px / 38px	Bunyan بنيان
displayMd	36px / 44px	Bunyan بنيان

Font families: `sans: System`, `arabic: System`, `mono: Courier`.
Weights: `regular: 400`, `medium: 500`, `semibold: 600`, `bold: 700`.
Letter spacing: `tight: -0.4`, `normal: 0`, `wide: 0.4`.

spacing

spacing.none	0px	
spacing.xxs	2px	
spacing.xs	4px	
spacing.sm	8px	■
spacing.md	12px	■
spacing.lg	16px	■
spacing.xl	20px	■
spacing.xxl	24px	■
spacing.xxxl	32px	■
spacing.giant	40px	■
spacing.huge	48px	■

radius

radius.none	0px	
radius.xs	2px	
radius.sm	4px	
radius.md	8px	
radius.lg	12px	
radius.xl	16px	
radius.pill	999px	

iconSize

iconSize.xs	12px
iconSize.sm	16px
iconSize.md	20px
iconSize.lg	24px
iconSize.xl	32px
iconSize.xxl	40px

componentHeight

componentHeight.xs	28px
componentHeight.sm	36px
componentHeight.md	44px
componentHeight.lg	52px
componentHeight.xl	60px

breakpoint

breakpoint.compact	0px
breakpoint.medium	600px
breakpoint.expanded	1024px
breakpoint.wide	1440px

opacity

opacity.invisible	0
opacity.subtle	0.08

borderWidth

<code>borderWidth.none</code>	0px
<code>borderWidth.thin</code>	1px
<code>borderWidth.medium</code>	2px
<code>borderWidth.thick</code>	3px

<code>opacity.muted</code>	0.4
<code>opacity.disabled</code>	0.5
<code>opacity.strong</code>	0.72
<code>opacity.opaque</code>	1

zIndex

<code>zIndex.base</code>	0
<code>zIndex.raised</code>	10
<code>zIndex.dropdown</code>	100
<code>zIndex.sticky</code>	200
<code>zIndex.overlay</code>	300
<code>zIndex.modal</code>	400
<code>zIndex.toast</code>	500
<code>zIndex.tooltip</code>	600

motion

<code>motion.duration.instant</code>	0ms
<code>motion.duration.fast</code>	120ms
<code>motion.duration.normal</code>	200ms
<code>motion.duration.slow</code>	320ms
<code>motion.easing.standard</code>	[0.2, 0, 0, 1]
<code>motion.easing.emphasized</code>	[0.2, 0, 0, 1]
<code>motion.easing.entrance</code>	[0, 0, 0, 1]
<code>motion.easing.exit</code>	[0.3, 0, 1, 1]

shadow

<code>shadow.none</code>	offset 0, radius 0, opacity 0, elevation 0
<code>shadow.sm</code>	offset 1, radius 2, opacity 0.08, elevation 1
<code>shadow.md</code>	offset 4, radius 8, opacity 0.12, elevation 4
<code>shadow.lg</code>	offset 10, radius 20, opacity 0.16, elevation 8

4 · Providers, hooks & utilities

ThemeProvider

PROP	TYPE	DEFAULT
<code>children</code> required	<code>React.ReactNode</code>	—
<code>initialPreference</code>	<code>'light' 'dark' 'black' 'system'</code>	<code>'system'</code>
<code>locale</code>	<code>string</code>	<code>'en'</code>
<code>direction</code>	<code>'ltr' 'rtl'</code>	—
<code>blackForSystemDark</code>	<code>boolean</code>	<code>false</code>

Locale direction: RTL for `ar`, `ckb`, `dv`, `fa`, `he`, `ps`, `sd`, `ug`, `ur`, `yi` (see `isRTLLocale`) or when the native layout is already RTL. With `initialPreference="system"` the provider follows OS appearance changes live.

useTheme()

```
const { theme, mode, preference, setPreference, direction, isRTL, locale } = useTheme();
```

PROP	TYPE	DEFAULT
<code>theme</code> required	<code>Theme</code>	—
<code>mode</code> required	<code>'light' 'dark' 'black'</code>	—
<code>preference</code> required	<code>'light' 'dark' 'black' 'system'</code>	—
<code>setPreference</code> required	<code>(preference: 'light' 'dark' 'black' 'system') => void</code>	—
<code>direction</code> required	<code>'ltr' 'rtl'</code>	—
<code>isRTL</code> required	<code>boolean</code>	—
<code>locale</code> required	<code>string</code>	—

Works without a provider too (falls back to the light theme, LTR, `en`).

useToast() / ToastProvider

```
const { showToast, dismissToast, dismissAll } = useToast();
const id = showToast({ message: 'Saved', tone: 'success', duration: 4000 });
```

Throws if used outside `ToastProvider`. See the Toast component page for options.

Theme objects

`lightTheme`, `darkTheme`, `blackTheme` and `themes[mode]` share one token contract (`Theme`): `mode`, `color`, `typography`, `spacing`, `radius`, `borderWidth`, `shadow`, `iconSize`, `componentHeight`, `breakpoint`, `motion`, `opacity`, `zIndex`.

Style utilities (`utilities/styles`)

EXPORT	SIGNATURE / NOTES
<code>logicalRow</code>	<code>logicalRow = (direction: Direction): TextStyle => ({ flexDirection: needsMirroring(direction) ? 'row-reverse' : 'row', })</code>
<code>logicalText</code>	<code>logicalText = (direction: Direction, align: LogicalAlign = 'start'): TextStyle => ({ textAlign: logicalTextAlign(direction, align), writingDirection: direction, })</code>
<code>logicalTextAlign</code>	<code>logicalTextAlign = (direction: Direction, align: LogicalAlign = 'start'): TextStyle['textAlign']</code>
<code>logicalAlignItems</code>	<code>logicalAlignItems = (direction: Direction, align: Exclude<LogicalAlign, 'center'>): TextStyle['alignItems']</code> <code>alignItems</code> value that places children at the logical start or end.
<code>needsMirroring</code>	<code>needsMirroring = (direction: Direction) => direction !== nativeDirection()</code> React Native already mirrors <code>row</code> and swaps <code>left</code> / <code>right</code> text alignment when the app runs natively in RTL. Bunyan's direction is provider-scoped, so we only mirror when the requested direction differs from the native one — otherwise RTL apps would be flipped twice and render left-to-right.
<code>nativeDirection</code>	<code>nativeDirection = (): Direction => (I18nManager.isRTL ? 'rtl' : 'ltr')</code> Direction the native layout engine is already using (set via <code>I18nManager</code>).
<code>heightForSize</code>	<code>heightForSize = (theme: Theme, size: ComponentSize) => ({ small: theme.componentHeight.sm, medium: theme.componentHeight.md, large: theme.componentHeight.lg, })[size]</code>
<code>horizontalPaddingForSize</code>	<code>horizontalPaddingForSize = (theme: Theme, size: ComponentSize) => ({ small: theme.spacing.md, medium: theme.spacing.lg, large: theme.spacing.xl, })[size]</code>
<code>iconSizeForComponent</code>	<code>iconSizeForComponent = (theme: Theme, size: ComponentSize) => ({ small: theme.iconSize.sm, medium: theme.iconSize.md, large: theme.iconSize.lg, })[size]</code>
<code>iconTokenForSize</code>	<code>iconTokenForSize = (size: ComponentSize) => (size === 'small' ? 'sm' : size === 'large' ? 'lg' : 'md') as 'sm' 'md' 'lg'</code> Icon size token name matching a component size.
<code>statusBorderColor</code>	<code>statusBorderColor = (theme: Theme, status: FeedbackStatus)</code>
<code>fontFamilyFor</code>	<code>fontFamilyFor = (theme: Theme, direction: Direction) => direction === 'rtl' ? theme.typography.fontFamily.arabic : theme.typography.fontFamily.sans</code>
<code>mergeIds</code>	<code>mergeIds = (...ids: Array<string undefined>) => ids.filter(Boolean).join('') undefined</code>
<code>isRTLLocale</code>	<code>isRTLLocale = (locale: string) => RTL_LOCALE.test(locale)</code>
<code>mergeThemedStyles</code>	<code>mergeThemedStyles = (...layers: AnyStyle[]): AnyStyle</code> Merges style layers, flattening <code>undefined</code> layers away. When any layer is a Pressable-style resolver the result is a resolver as well, so interaction state keeps working through the themed wrapper.
<code>DIRECTIONAL_ICONS</code>	<code>DIRECTIONAL_ICONS: ReadonlySet<IconName> = new Set<IconName>(['backspace', 'chevron-left', 'chevron-right'])</code> Icons whose meaning depends on reading direction and should mirror in RTL.

Typography & icons

1 Text

Semantic body, label, caption, and code text.

```
import { Text } from '@bunyan/design-system';
```

- Variants & states**
- Accessibility**
- Example**
- Do/don't**
- Edge cases**
- body**, **bodySmall**, **caption**, **label**, **code**; tone and weight APIs; inherits native selection and truncation states.
- Font scaling is enabled with a 2× multiplier; direction follows the provider.
- `<Text variant="bodySmall" tone="secondary">Updated today</Text>`.
- Use tone names; do not pass raw color or font styles.
- Use `numberOfLines` for constrained layouts and test at maximum font size.

TextProps

PROP	TYPE	DEFAULT
variant	'body' 'bodySmall' 'caption' 'label' 'code'	'body'
tone	'primary' 'secondary' 'tertiary' 'inverse' 'link' 'error' 'success'	'primary'
align	'start' 'center' 'end'	'start'
weight	'regular' 'medium' 'semibold' 'bold'	'regular'

Also accepts: `RNTextProps` (native React Native props).

variant

The quick brown fox

The quick brown fox

The quick brown fox

The quick brown fox

const x = 1;

body

bodySmall

caption

label

code

tone

Sample

Sample

Sample

Sample

Sample

Sample

primary

secondary

tertiary

link

error

success

Sample

inverse

weight

Weight

Weight

Weight

Weight

regular

medium

semibold

bold

align

Aligned

Aligned

Aligned

start

center

end

All variants — light theme

variant

The quick brown fox

The quick brown fox

The quick brown fox

The quick brown fox

const x = 1;

body

bodySmall

caption

label

code

tone

Sample

Sample

Sample

Sample

Sample

Sample

primary

secondary

tertiary

link

error

success

Sample

inverse

weight

Weight

Weight

Weight

Weight

regular

medium

semibold

bold

align

Aligned

Aligned

Aligned

start

center

end

Dark theme

variant

نص تجريبي للعرض

نص تجريبي للعرض

نص تجريبي للعرض

نص تجريبي للعرض

نص تجريبي للعرض

نص تجريبي للعرض

body

bodySmall

caption

label

code

code

tone

عينة

عينة

عينة

عينة

عينة

عينة

primary

secondary

tertiary

link

error

success

عينة

inverse

weight

الوزن

الوزن

الوزن

الوزن

regular

medium

semibold

bold

align

محاذة

محاذة

محاذة

start

center

end

Arabic (ar-SA) · RTL

2 Heading

Establishes an accessible content hierarchy.

```
import { Heading } from '@bunyan/design-system';
```

- Variants & states**
- Levels 1–6 map to the type scale.
- Accessibility**
- Uses the header role and web `aria-level`.
- Example**
- `<Heading level={2}>Account summary</Heading>`.
- Do/don't**
- Keep levels sequential; do not choose a level only for visual size.
- Edge cases**
- Long localized headings should wrap; avoid truncating critical titles.

HeadingProps

PROP	TYPE	DEFAULT
<code>level</code>	1 2 3 4 5 6	2
<code>tone</code>	'primary' 'secondary' 'tertiary' 'inverse' 'link' 'error' 'success'	'primary'
<code>align</code>	'start' 'center' 'end'	'start'

Also accepts: `RNTextProps` (native React Native props).

level

Heading level 1

Heading level 2

Heading level 3

Heading level 4

Heading level 5

Heading level 6

tone

Title

Title

Title

Title

Title

primary

secondary

link

error

success

All variants — light theme

level

Heading level 1

Heading level 2

Heading level 3

Heading level 4

Heading level 5

Heading level 6

tone

Title

primary

Title

secondary

Title

link

Title

error

Title

success

Dark theme

level

عنوان المستوى 1

عنوان المستوى 2

عنوان المستوى 3

عنوان المستوى 4

عنوان المستوى 5

عنوان المستوى 6

tone

عنوان

success

عنوان

error

عنوان

link

عنوان

secondary

عنوان

primary

Arabic (ar-SA) · RTL

3 Icon

Renders the controlled Bunyan SVG icon set.

```
import { Icon } from '@bunyan/design-system';
```

- Variants & states**
- Six token sizes and eight semantic tones.
- Accessibility**
- Decorative icons (no `accessibilityLabel`) are hidden from assistive tech; labelled icons get the image role.
- Example**
- `<Icon name="chevron-right" mirroredInRTL />`.
- Do/don't**
- Prefer semantic tone; use `color` only for approved brand exceptions.
- Edge cases**
- Directional icons (chevron-left/right, backspace) mirror automatically in RTL; pass `mirroredInRTL` to override.

IconProps

PROP	TYPE	DEFAULT
<code>name</code> required	<code>IconName</code>	—
<code>size</code>	<code>'xs' 'sm' 'md' 'lg' 'xl' 'xxl'</code>	<code>'md'</code>
<code>tone</code>	<code>'primary' 'secondary' 'tertiary' 'inverse' 'success' 'warning' 'error' 'information'</code>	<code>'primary'</code>
<code>color</code>	<code>string</code>	—
<code>mirroredInRTL</code>	<code>boolean</code> Mirror in RTL. Defaults to <code>true</code> for directional icons only.	<code>DIRECTIONAL_ICONS.has(name)</code>
<code>accessibilityLabel</code>	<code>string</code>	—
<code>testID</code>	<code>string</code>	—

name


alert-circle


backspace


calendar


check


chevron-down


chevron-left


chevron-right


close


eye


eye-off


info


search


success


warning


error


user

size


xs


sm


md


lg


xl


xxl

tone


primary


secondary


tertiary


success


warning


error


information

All variants — light theme

name


alert-circle


backspace


calendar


check


chevron-down


chevron-left


chevron-right


close


eye


eye-off


info


search


success


warning


error


user

size


xs


sm


md


lg


xl


xxl

tone


primary


secondary


tertiary


success


warning


error


information

Dark theme

							name
							
chevron-left	chevron-down	check	calendar	backspace	alert-circle		
							
search	info	eye-off	eye	close	chevron-right		
							
		user	error	warning	success		
							size
							
xxl	xl	lg	md	sm	xs		
							tone
							
error	warning	success	tertiary	secondary	primary		
							information

Arabic (ar-SA) · RTL

Actions

4 Button

Triggers an action, or — with `variant="link"` / `href` — navigates.

```
import { Button } from '@bunyan/design-system';
```

- Variants & states

`primary`, `secondary`, `outline`, `ghost`, `danger`, `link`; small/medium/large; pressed, disabled, loading, busy.
- Accessibility

Button role (link role for `variant="link"` or when `href` is set); busy/disabled states are automatic; external links announce `externalHint`; compact sizes and links get hitSlop for a 44 pt target.
- Example

`<Button variant="primary" size="large" fullWidth>Continue</Button>` · `<Button variant="link" href="https://bunyan.sa" external>Help centre</Button>`.
- Do/don't

Use one primary action per region; use `variant="link"` (not `ghost`) for navigation inside text-heavy layouts.
- Edge cases

Loading disables duplicate submission; `href` opens after `onPress` unless the event is `preventDefault` -ed; `openURL` failures go to `onOpenError`.

ButtonProps

PROP	TYPE	DEFAULT
<code>children</code> required	<code>React.ReactNode</code>	—
<code>variant</code>	<code>'primary' 'secondary' 'outline' 'ghost' 'danger' 'link'</code> <code>link</code> renders inline link text (no container); every other variant is a filled/outlined button.	<code>'primary'</code>
<code>size</code>	<code>'small' 'medium' 'large'</code>	<code>'medium'</code>
<code>loading</code>	<code>boolean</code>	<code>false</code>
<code>fullWidth</code>	<code>boolean</code>	<code>false</code>
<code>leadingIcon</code>	<code>IconName</code>	—
<code>trailingIcon</code>	<code>IconName</code>	—
<code>href</code>	<code>string</code> URL opened with <code>Linking.openURL</code> after <code>onPress</code> (unless the event was <code>preventDefault</code> -ed). Gives the control the link role.	—
<code>external</code>	<code>boolean</code> Marks the destination as outside the app: shows ↗ and announces <code>externalHint</code> .	<code>false</code>
<code>externalHint</code>	<code>string</code> Screen-reader hint for external destinations.	<code>'Opens in another app'</code>
<code>onOpenError</code>	<code>(error: unknown) => void</code> Called when <code>href</code> cannot be opened.	—

Also accepts: `Omit<PressableProps, 'children' | 'style'>` (native React Native props).

variant="primary"

Continue small	Continue medium	Continue large	✓ Next > leading + trailing	Saving loading	Disabled disabled
--	---	--	--	---	---

variant="secondary"

Continue small	Continue medium	Continue large	✓ Next > leading + trailing	Saving loading	Disabled disabled
--	---	--	--	---	---

variant="outline"

Continue small	Continue medium	Continue large	✓ Next > leading + trailing	Saving loading	Disabled disabled
--	---	--	--	---	---

variant="ghost"

Continue small	Continue medium	Continue large	✓ Next > leading + trailing	Saving loading	Disabled disabled
--	---	--	--	---	---

variant="danger"

Continue small	Continue medium	Continue large	✓ Next > leading + trailing	Saving loading	Disabled disabled
--	---	--	--	---	---

variant="link"

Continue small	Continue medium	Continue large	✓ Next > leading + trailing	Saving loading	Disabled disabled
--	---	--	--	--	---

variant="link" with href

Terms of service href	Help centre ↗ href + external	Open website ↗ variant="primary" + href
---	---	---

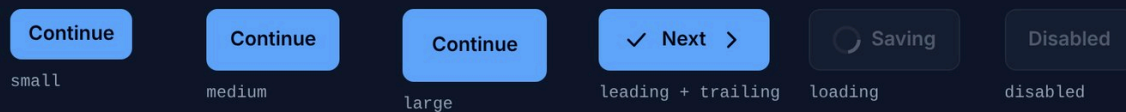
fullWidth

Create account

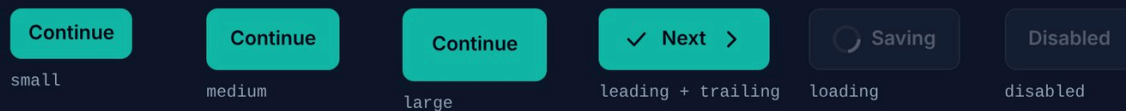
fullWidth

All variants — light theme

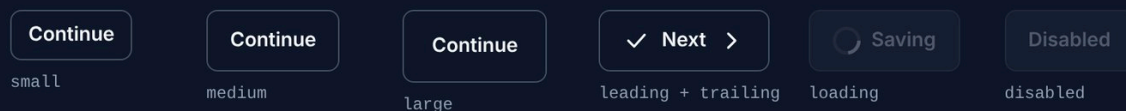
variant="primary"



variant="secondary"



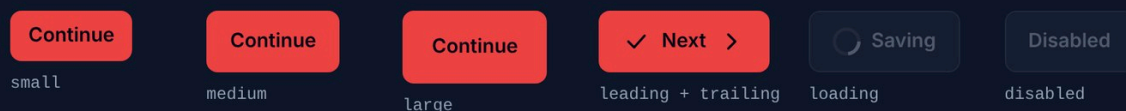
variant="outline"



variant="ghost"



variant="danger"



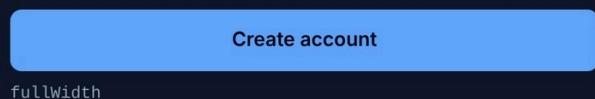
variant="link"



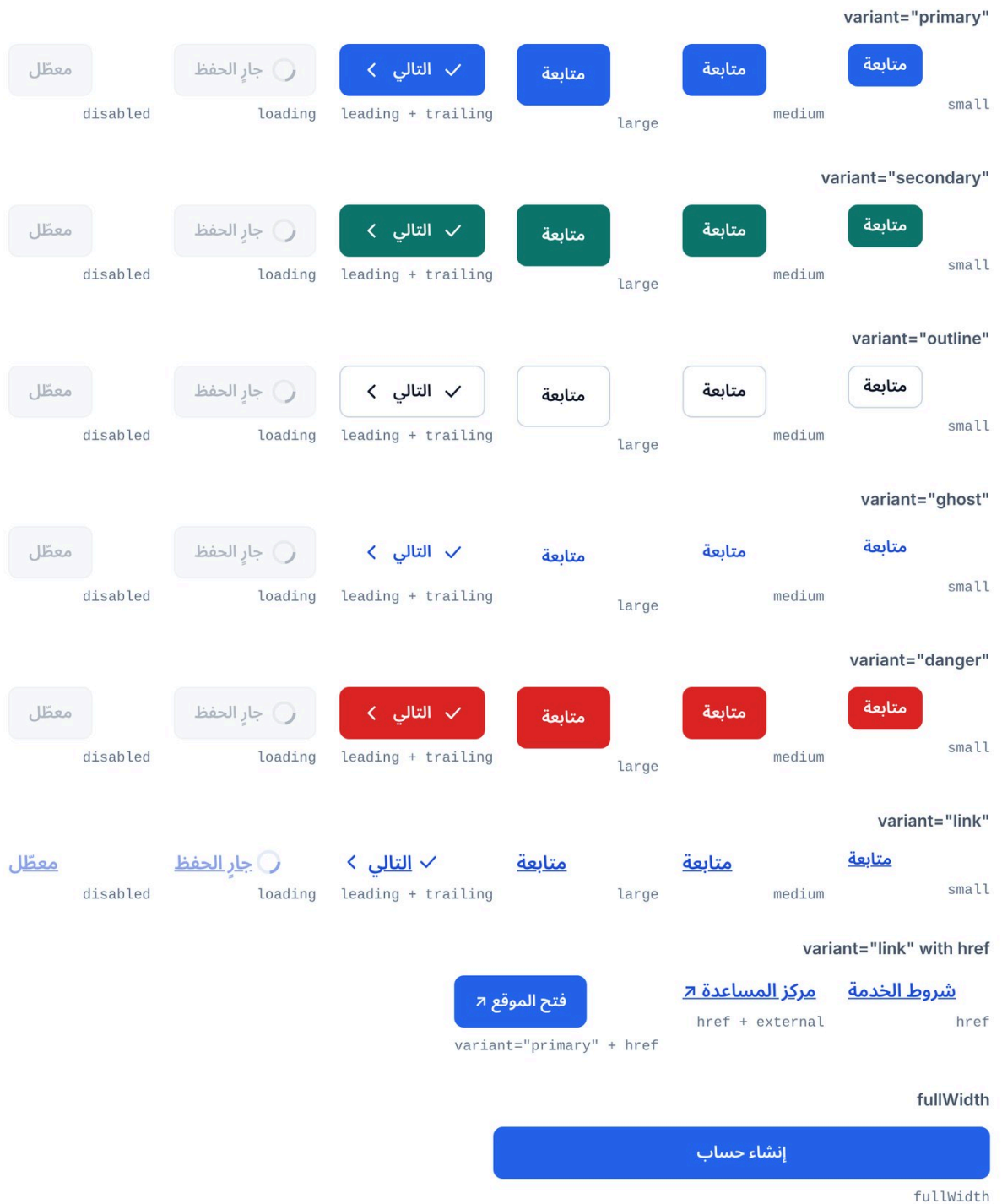
variant="link" with href



fullWidth



Dark theme



5 IconButton

Compact icon-only action.

```
import { IconButton } from '@bunyan/design-system';
```

- Variants & states**
- Filled, outline, ghost; small/medium/large; pressed, selected, disabled.
- Accessibility**
- A text label is mandatory despite the visual icon-only treatment.
- Example**
- `<IconButton icon="close" accessibilityLabel="Close dialog" />.`
- Do/don't**
- Use familiar icons; do not rely on color alone for selection.
- Edge cases**
- Add surrounding space when several icon buttons form a toolbar.

IconButtonProps

PROP	TYPE	DEFAULT
<code>icon</code> required	<code>IconName</code>	—
<code>accessibilityLabel</code> required	<code>string</code> Overrides the text that's read by the screen reader when the user interacts with the element. By default, the label is constructed by traversing all the children and accumulating all the Text nodes separated by space.	—
<code>variant</code>	<code>'filled' 'outline' 'ghost'</code>	<code>'ghost'</code>
<code>size</code>	<code>'small' 'medium' 'large'</code>	<code>'medium'</code>
<code>tone</code>	<code>'primary' 'secondary' 'tertiary' 'inverse' 'success' 'warning' 'error' 'information'</code>	<code>'primary'</code>
<code>selected</code>	<code>boolean</code>	<code>false</code>
<code>mirroredInRTL</code>	<code>boolean</code> Mirror the glyph in RTL. Defaults to automatic (directional icons only).	—

Also accepts: `Omit<PressableProps, 'children' | 'style'>` (native React Native props).

variant="ghost"

small

medium

large

selected

disabled

tone=error

variant="outline"

small

medium

large

selected

disabled

tone=error

variant="filled"

small

medium

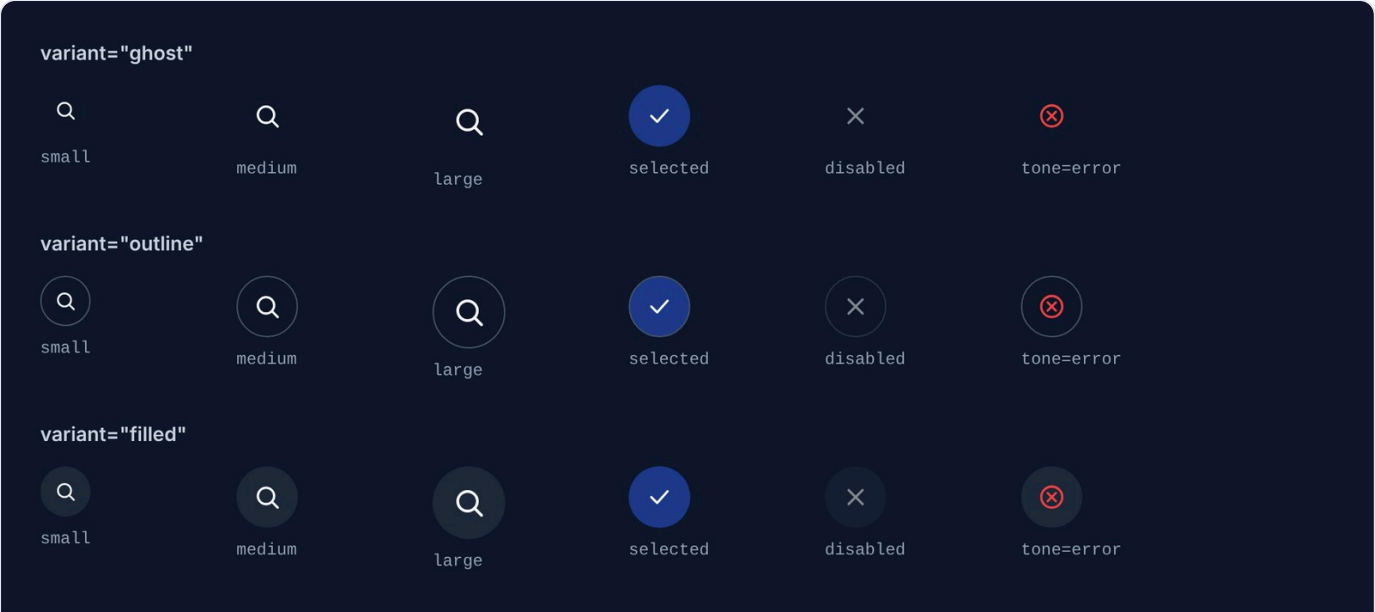
large

selected

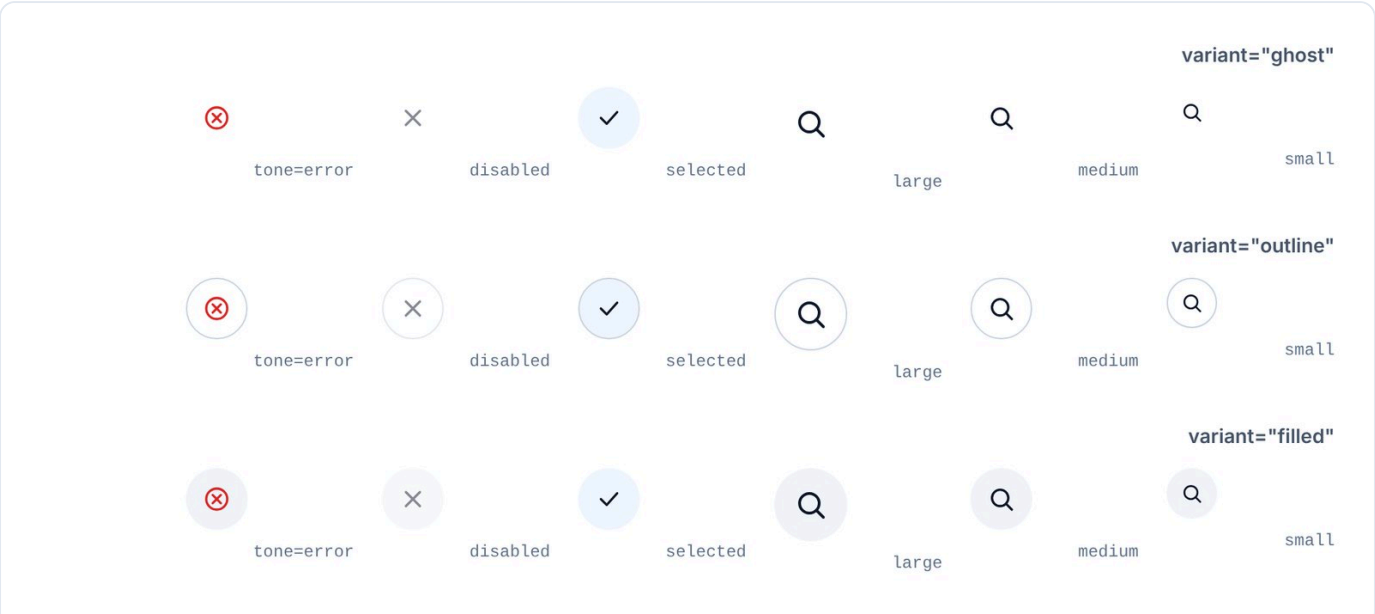
disabled

tone=error

All variants — light theme



Dark theme



Arabic (ar-SA) · RTL

6 Link

Inline navigation text. `Link` is an alias of `<Button variant="link" />` and accepts every `Button` prop except `variant`.

```
import { Link } from '@bunyan/design-system';
```

- Variants & states

small/medium/large text; pressed, disabled, loading.
- Accessibility

Uses the link role; external links show ↗ and announce `externalHint`. `openURL` failures go to `onOpenError`.
- Example

`<Link href="https://example.com" external>Privacy policy</Link>` (same as `<Button variant="link" ...>`).
- Do/don't

Use descriptive text; do not label links "click here."
- Edge cases

Kept for convenience and backwards compatibility; styling lives in `Button`, so both stay in sync.

LinkProps

PROP	TYPE	DEFAULT
<code>children</code> required	<code>React.ReactNode</code>	—
<code>size</code>	<code>'small' 'medium' 'large'</code>	—
<code>loading</code>	<code>boolean</code>	—
<code>fullWidth</code>	<code>boolean</code>	—
<code>leadingIcon</code>	<code>IconName</code>	—
<code>trailingIcon</code>	<code>IconName</code>	—
<code>href</code>	<code>string</code> URL opened with <code>Linking.openURL</code> after <code>onPress</code> (unless the event was <code>preventDefault</code> -ed). Gives the control the link role.	—
<code>external</code>	<code>boolean</code> Marks the destination as outside the app: shows ↗ and announces <code>externalHint</code> .	—
<code>externalHint</code>	<code>string</code> Screen-reader hint for external destinations.	—
<code>onOpenError</code>	<code>(error: unknown) => void</code> Called when <code>href</code> cannot be opened.	—

Also accepts: `Omit<ButtonProps>` (native React Native props).

Terms of service

Help centre ↗

Unavailable

Small link

i

 Learn more

⌂

 Loading

default

external

disabled

size="small"

leadingIcon

loading

All variants — light theme

Terms of service

Help centre ↗

Unavailable

Small link

i

 Learn more

⌂

 Loading

default

external

disabled

size="small"

leadingIcon

loading

Dark theme

جارٍ التحميل

loading

اعرف المزيد ⓘ

leadingIcon

رابط صغير

size="small"

غير متاح

disabled

مركز المساعدة 7

external

شروط الخدمة

default

Arabic (ar-SA) · RTL

Form controls

7 Input

Single-line text entry primitive.

```
import { Input } from '@bunyan/design-system';
```

- Variants & states** Small/medium/large; default/error/success; focused and disabled.
- Accessibility** Supports native labelling props and announces invalid state with `aria-invalid`.
- Example** `<Input keyboardType="email-address" status="error" />`.
- Do/don't** Wrap in `FormField`; do not use placeholder text as the only label.
- Edge cases** Controlled inputs require `onChangeText`; custom trailing controls must have labels.

InputProps

PROP	TYPE	DEFAULT
size	'small' 'medium' 'large'	'medium'
status	'default' 'error' 'success'	'default'
leadingIcon	IconName	—
trailing	React.ReactNode	—
containerStyle	Pick<ViewStyle, 'flex' 'width' 'maxWidth' 'minWidth' 'alignSelf'>	—

Also accepts: `Omit<TextInputProps, 'style'>` (native React Native props).

size

Account name

small

Account name

medium

Account name

large

status

Bunyan Holding

default

Bunyan Holding

error

Bunyan Holding

success

adornments & states

👤 Username

leadingIcon

0.00

SAR

trailing

Read only

editable={false}

All variants — light theme

size

Account name

Account name

Account name

small

medium

large

status

Bunyan Holding

Bunyan Holding

Bunyan Holding

default

error

success

adornments & states

⤵

Username

0.00SAR

Read only

leadingIcon

trailing

editable={false}

Dark theme

size

اسم الحساب

اسم الحساب

اسم الحساب

large

medium

small

status

بيان القابضة

بيان القابضة

بيان القابضة

success

error

default

adornments & states

للقراءة فقط

SAR0.00

اسم المستخدم ⤵

editable={false}

trailing

leadingIcon

Arabic (ar-SA) · RTL

8 PasswordInput

Secure text entry with visibility control.

```
import { PasswordInput } from '@bunyan/design-system';
```

- Variants & states**
- Inherits Input states; hidden and visible states.
- Accessibility**
- Visibility button announces its current action.
- Example**
- `<PasswordInput accessibilityLabel="Password" />.`
- Do/don't**
- Allow password managers; do not block paste.
- Edge cases**
- Platform keyboards may briefly retain suggestions after visibility changes.

PasswordInputProps

PROP	TYPE	DEFAULT
status	'default' 'error' 'success'	—
size	'small' 'medium' 'large'	—
leadingIcon	IconName	—
containerStyle	Pick<ViewStyle, 'flex' 'width' 'maxWidth' 'minWidth' 'alignSelf'>	—
showPasswordLabel	string	'Show password'
hidePasswordLabel	string	'Hide password'

Also accepts: `Omit<InputProps` (native React Native props).

.....

👁

hidden

...

👁

status=error

.....

👁

disabled

All variants — light theme

.....

👁

hidden

...

👁

status=error

.....

👁

disabled

Dark theme

👁

.....

disabled

👁

...

status=error

👁

.....

hidden

Arabic (ar-SA) · RTL

9 SearchInput

Search query entry with search and clear affordances.

```
import { SearchInput } from '@bunyan/design-system';
```

- Variants & states**
- Inherits Input; clear action appears when `value` is non-empty.
- Accessibility**
- Search return key and labelled clear button.
- Example**
- `<SearchInput value={query} onChangeText={setQuery} onClear={() => setQuery('')} />.`
- Do/don't**
- Debounce network requests outside the component; do not clear without user action.
- Edge cases**
- Works controlled or uncontrolled; Clear calls `onChangeText('')` and `onClear`, then refocuses the field.

SearchInputProps

PROP	TYPE	DEFAULT
<code>clearLabel</code>	<code>string</code>	<code>'Clear search'</code>
<code>onClear</code>	<code>() => void</code>	—
<code>status</code>	<code>'default' 'error' 'success'</code>	—
<code>size</code>	<code>'small' 'medium' 'large'</code>	—
<code>containerStyle</code>	<code>Pick<ViewStyle, 'flex' 'width' 'maxWidth' 'minWidth' 'alignSelf'></code>	—

Also accepts: `Omit<InputProps, 'leadingIcon' | 'trailing'>` (native React Native props).

Q Search accounts

empty

Q report

with value (clear button)

Q Search

size=small

All variants — light theme

Q Search accounts

empty

Q report

with value (clear button)

Q Search

size=small

Dark theme

Q بحث

size=small

× تقرير

with value (clear button)

Q ابحث في الحسابات

empty

Arabic (ar-SA) · RTL

10 TextArea

Multi-line text entry.

```
import { TextArea } from '@bunyan/design-system';
```

- Variants & states** Default/error/success; focused, disabled, character limit.
- Accessibility** Exposes invalid and disabled semantics; counter remains visible.
- Example** `<TextArea minRows={6} maxLength={500} />`.
- Do/don't** Set realistic limits; do not resize based on every keystroke in dense forms.
- Edge cases** Very large font settings increase the rendered minimum height.

TextAreaProps

PROP	TYPE	DEFAULT
status	'default' 'error' 'success'	'default'
minRows	number	4
maxLength	number	—
	Limits the maximum number of characters that can be entered. Use this instead of implementing the logic in JS to avoid flicker.	
showCounter	boolean	Boolean(maxLength)

Also accepts: `Omit<TextInputProps, 'style' | 'multiline'>` (native React Native props).

Describe the issue

default

Delivery delayed

16/120

maxLength counter

!

status=error

Locked note

disabled

All variants — light theme

Describe the issue

default

Delivery delayed

16/120

maxLength counter

!

status=error

Locked note

disabled

Dark theme

!

status=error

تأخر التوصيل

12/120

maxLength counter

صف المشكلة

default

ملاحظة مقفلة

disabled

Arabic (ar-SA) · RTL

11 FormField

Groups label, description, control, error, and success feedback.

```
import { FormField } from '@bunyan/design-system';
```

Variants & states Required/optional, error/success.

Accessibility Render props provide stable IDs for label and description relationships.

Example

```
<FormField label="Email">{ids => <Input accessibilityLabelledBy={ids.labelId} />}</FormField>.
```

Do/don't Show one actionable error; do not encode required state only with an asterisk.

Edge cases Localize `optionalLabel` and all feedback strings.

FormFieldProps

PROP	TYPE	DEFAULT
<code>label</code> required	<code>string</code>	—
<code>children</code> required	<code>React.ReactNode ((props: FormFieldRenderProps) => React.ReactNode)</code>	—
<code>description</code>	<code>string</code>	—
<code>error</code>	<code>string</code>	—
<code>success</code>	<code>string</code>	—
<code>required</code>	<code>boolean</code>	<code>false</code>
<code>optionalLabel</code>	<code>string</code>	<code>'Optional'</code>

FormFieldRenderProps

PROP	TYPE	DEFAULT
<code>inputId</code> required	<code>string</code>	—
<code>labelId</code> required	<code>string</code>	—
<code>descriptionId</code>	<code>string</code>	—
<code>errorId</code>	<code>string</code>	—

Email *

We never share it.

required + description

Nickname (Optional)

optional

IBAN *

Enter a valid IBAN

error

Username *

Available

success

All variants — light theme

Email *

We never share it.

name@company.com

required + description

Nickname (Optional)

optional

IBAN *

SA00

Enter a valid IBAN

error

Username *

faisal

Available

success

Dark theme

الاسم المستعار (اختياري)

optional

البريد الإلكتروني *

لن نشاركه أبدًا.

name@company.com

required + description

اسم المستخدم *

faisal

متاح

success

الآيبان *

أدخل آيبان صالحًا

SA00

error

Arabic (ar-SA) · RTL

12 Checkbox

Selects zero or more independent options.

```
import { Checkbox } from '@bunyan/design-system';
```

Variants & states Checked, unchecked, mixed, disabled, error.

Accessibility Checkbox role and true/false/mixed state.

Example `<Checkbox checked={accepted} onChange={setAccepted} label="Accept terms" />.`

Do/don't Use for independent choices; do not use for mutually exclusive options.

Edge cases Pressing a mixed (indeterminate) checkbox always reports `true`.

CheckboxProps

PROP	TYPE	DEFAULT
<code>checked</code> required	<code>boolean</code>	—
<code>onChange</code> required	<code>(checked: boolean) => void</code>	—
<code>label</code> required	<code>string</code>	—
<code>description</code>	<code>string</code>	—
<code>disabled</code>	<code>boolean</code>	<code>false</code>
<code>indeterminate</code>	<code>boolean</code>	<code>false</code>
<code>error</code>	<code>boolean</code>	<code>false</code>

☐ Accept terms
unchecked

☒ Accept terms
checked

☐ Select all
indeterminate

☒ Notifications
Email and SMS
description

☐ Required
error

☒ Locked
disabled

All variants — light theme

☐ Accept terms
unchecked

☒ Accept terms
checked

☐ Select all
indeterminate

☒ Notifications
Email and SMS
description

☐ Required
error

☒ Locked
disabled

Dark theme

☐ تحديد الكل
indeterminate

☒ أوافق على الشروط
checked

☐ أوافق على الشروط
unchecked

☒ مقفل
disabled

☐ مطلوب
error

☒ الإشعارات
البريد والرسائل
description

Arabic (ar-SA) · RTL

13 Radio

Selects exactly one option in a set.

```
import { Radio } from '@bunyan/design-system';
```

Variants & states Selected, unselected, disabled.

Accessibility Radio role, checked state, and optional value.

Example `<Radio selected={plan === 'pro'} onSelect={() => setPlan('pro')} label="Pro" /> .`

Do/don't Present radios as a labelled group; do not offer only one radio.

Edge cases Re-pressing the selected radio is a no-op; product code owns arrow-key group navigation on web.

RadioProps

PROP	TYPE	DEFAULT
<code>selected</code> required	boolean	—
<code>onSelect</code> required	<code>() => void</code>	—
<code>label</code> required	string	—
<code>description</code>	string	—
<code>disabled</code>	boolean	<code>false</code>
<code>value</code>	string	—

☒ Standard plan
selected

☐ Premium plan
unselected

☐ Enterprise
For 50+ seats
description

☐ Legacy
disabled

All variants — light theme

☒ Standard plan
selected

☐ Premium plan
unselected

☐ Enterprise
For 50+ seats
description

☐ Legacy
disabled

Dark theme

المؤسسات
لأكثر من 50 مستخدمًا
description

☐ الخطة المميزة
unselected

☒ الخطة القياسية
selected

قديم
disabled

Arabic (ar-SA) · RTL

14 Switch

Immediately toggles a setting.

```
import { Switch } from '@bunyan/design-system';
```

- Variants & states

On, off, disabled.
- Accessibility

Native switch role and checked state.
- Example

`<Switch value={enabled} onChange={setEnabled} label="Notifications" />.`
- Do/don't

Use for immediate effects; use Checkbox when submission is required.
- Edge cases

Persist failures should restore the prior value and show feedback.

SwitchProps

PROP	TYPE	DEFAULT
<code>value</code> required	<code>boolean</code>	—
<code>onChange</code> required	<code>(value: boolean) => void</code>	—
<code>label</code> required	<code>string</code>	—
<code>description</code>	<code>string</code>	—
<code>disabled</code>	<code>boolean</code>	<code>false</code>

Notifications

on

Dark mode

off

Biometrics

Use Face ID to sign in
description

Beta features

disabled

All variants — light theme

Notifications

on

Dark mode

off

Biometrics

Use Face ID to sign in
description

Beta features

disabled

Dark theme

الوضع الداكن

off

الإشعارات

on

ميزات تجريبية

disabled

البصمة

استخدم بصمة الوجه للدخول
description

Arabic (ar-SA) · RTL

15 Select

Chooses one value from a list.

```
import { Select } from '@bunyan/design-system';
```

Variants & states Small/medium/large; expanded, disabled, error, selected, empty search.

Accessibility Combobox role, current value, expanded and invalid state.

Example `<Select options={countries} value={country} onValueChange={setCountry} searchable />.`

Do/don't Use search for long lists; do not use for fewer than three obvious choices.

Edge cases Disabled options remain visible; missing values show the placeholder.

SelectProps

PROP	TYPE	DEFAULT
<code>options</code> required	<code>readonly SelectOption[]</code>	—
<code>value</code>	<code>string</code>	—
<code>onValueChange</code> required	<code>(value: string) => void</code>	—
<code>placeholder</code>	<code>string</code>	<code>'Select an option'</code>
<code>title</code>	<code>string</code>	<code>'Select an option'</code>
<code>disabled</code>	<code>boolean</code>	<code>false</code>
<code>status</code>	<code>'default' 'error' 'success'</code>	<code>'default'</code>
<code>size</code>	<code>'small' 'medium' 'large'</code>	<code>'medium'</code>
<code>searchable</code>	<code>boolean</code>	<code>false</code>
<code>searchPlaceholder</code>	<code>string</code>	<code>'Search options'</code>
<code>emptyMessage</code>	<code>string</code>	<code>'No options found'</code>
<code>accessibilityLabel</code>	<code>string</code>	—

SelectOption

PROP	TYPE	DEFAULT
<code>value</code> required	<code>string</code>	—
<code>label</code> required	<code>string</code>	—
<code>description</code>	<code>string</code>	—
<code>disabled</code>	<code>boolean</code>	—

state

Select a city

▼

placeholder

Riyadh

▼

selected

Required

▼

status=error

Jeddah

▼

disabled

size

Dammam

▼

small

Dammam

▼

medium

Dammam

▼

large

All variants — light theme

state

Select a city

▼

placeholder

Riyadh

▼

selected

Required

▼

status=error

Jeddah

▼

disabled

size

Dammam

▼

small

Dammam

▼

medium

Dammam

▼

large

Dark theme

state

▼

مطلوب

status=error

▼

الرياض

selected

▼

اختر مدينة

placeholder

▼

جدة

disabled

size

▼

الدمام

large

▼

الدمام

medium

▼

الدمام

small

Arabic (ar-SA) · RTL

Open (searchable) sheet

Jeddah

Select a city

Q Search cities

Riyadh
Central region

Jeddah
Western region

Dammam
Eastern region

Abha

Light

Jeddah

Select a city

Q Search cities

Riyadh
Central region

Jeddah
Western region

Dammam
Eastern region

Abha

Dark

اختر مدينة

ابحث عن مدينة

الرياض
المنطقة الوسطى

جدة
المنطقة الغربية

الدمام
المنطقة الشرقية

أبها

Arabic · RTL

16 DatePicker

Selects a calendar date consistently across native platforms.

```
import { DatePicker } from '@bunyan/design-system';
```

Variants & states Empty, selected, open, disabled, error.

Accessibility Pressable trigger with button role, current value and expanded state; clear is a separate labelled button.

Example `<DatePicker value={date} onChange={setDate} maximumDate={new Date()} />.`

Do/don't Pass a locale and valid bounds; do not parse display text manually.

Edge cases Label uses the Gregorian calendar by default (ar-SA would otherwise show Hijri); override with `formatOptions`. Value is clamped to min/max on confirm.

DatePickerProps

PROP	TYPE	DEFAULT
<code>value</code>	<code>Date</code>	—
<code>onChange</code> required	<code>(date: Date undefined) => void</code>	—
<code>minimumDate</code>	<code>Date</code>	—
<code>maximumDate</code>	<code>Date</code>	—
<code>locale</code>	<code>string</code>	—
<code>placeholder</code>	<code>string</code>	<code>'Select date'</code>
<code>title</code>	<code>string</code>	—
<code>disabled</code>	<code>boolean</code>	<code>false</code>
<code>status</code>	<code>'default' 'error' 'success'</code>	<code>'default'</code>
<code>size</code>	<code>'small' 'medium' 'large'</code>	<code>'medium'</code>
<code>accessibilityLabel</code>	<code>string</code>	<code>'Date'</code>
<code>clearable</code>	<code>boolean</code>	<code>true</code>
<code>clearLabel</code>	<code>string</code>	<code>'Clear date'</code>
<code>confirmLabel</code>	<code>string</code>	<code>'Confirm'</code>
<code>formatOptions</code>	<code>Intl.DateTimeFormatOptions</code> Display format. Defaults to the Gregorian calendar so the label matches the (Gregorian) native picker — <code>ar-SA</code> would otherwise format as Hijri. Pass <code>{ calendar: 'islamic-umalqura' }</code> to opt into Hijri display.	—
<code>testID</code>	<code>string</code>	—

Select date

empty

Oct 3, 2026

×

with value (clearable)

Date of birth

status=error

Jan 15, 2026

disabled

All variants — light theme

Select date

empty

Oct 3, 2026

×

with value (clearable)

Date of birth

status=error

Jan 15, 2026

disabled

Dark theme

تاريخ الميلاد

status=error

٣ أكتوبر ٢٠٢٦

×

with value (clearable)

اختر التاريخ

empty

١٥ يناير ٢٠٢٦

disabled

Arabic (ar-SA) · RTL

Open picker sheet (iOS / web presentation)

Oct 3, 2026

Delivery date

October 2026

S	M	T	W	T	F	S
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

Confirm

Light

Oct 3, 2026

Delivery date

October 2026

S	M	T	W	T	F	S
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

Confirm

Dark

٣ أكتوبر ٢٠٢٦

تاريخ التسليم

October 2026

S	M	T	W	T	F	S
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

تأكيد

Arabic · RTL

Data display

17 Badge

Displays compact read-only status or metadata.

```
import { Badge } from '@bunyan/design-system';
```

- Variants & states** Neutral, primary, success, warning, error, information; small/medium.
- Accessibility** Text remains the source of meaning; color is supplemental.
- Example** `<Badge tone="success">Approved</Badge>`.
- Do/don't** Keep copy short; do not make badges interactive.
- Edge cases** Long localized status text may wrap; use a Chip if interaction is needed.

BadgeProps

PROP	TYPE	DEFAULT
<code>children</code> required	<code>React.ReactNode</code>	—
<code>tone</code>	<code>'neutral' 'primary' 'success' 'warning' 'error' 'information'</code>	<code>'neutral'</code>
<code>size</code>	<code>'small' 'medium'</code>	<code>'medium'</code>
<code>accessibilityLabel</code>	<code>string</code>	—

size="medium"

Neutral
neutral

Primary
primary

Success
success

Warning
warning

Error
error

Information
information

size="small"

Neutral
neutral

Primary
primary

Success
success

Warning
warning

Error
error

Information
information

All variants — light theme

size="medium"

Neutral
neutral

Primary
primary

Success
success

Warning
warning

Error
error

Information
information

size="small"

Neutral
neutral

Primary
primary

Success
success

Warning
warning

Error
error

Information
information

Dark theme

حالة

information

حالة

error

حالة

warning

حالة

success

حالة

primary

حالة

neutral

size="medium"

حالة

information

حالة

error

حالة

warning

حالة

success

حالة

primary

حالة

neutral

size="small"

Arabic (ar-SA) · RTL

18 Chip

Represents a filter, selection, or removable value.

```
import { Chip } from '@bunyan/design-system';
```

- Variants & states**
- Default, selected, pressed, disabled, removable.
- Accessibility**
- Label and remove are sibling buttons (never nested), each with its own label and state.
- Example**
- `<Chip label="Finance" selected onRemove={removeFinance} />.`
- Do/don't**
- Use concise nouns; do not nest arbitrary controls inside a chip.
- Edge cases**
- Removing a selected filter should update focus predictably.

ChipProps

PROP	TYPE	DEFAULT
label required	string	—
selected	boolean	false
disabled	boolean	false
onPress	() => void	—
onRemove	() => void	—
accessibilityLabel	string	—
removeLabel	(label: string) => string	value => `Remove \${value}`
testID	string	—

Enterprise

Enterprise

Riyadh ×

Riyadh ×

Archived

defaultselectedremovableselected + removabledisabled

All variants — light theme

Enterprise

Enterprise

Riyadh ×

Riyadh ×

Archived

defaultselectedremovableselected + removabledisabled

Dark theme

مؤرشف

× الرياض

× الرياض

المؤسسات

المؤسسات

disabledselected + removableremovableselecteddefault

Arabic (ar-SA) · RTL

19 Avatar

Identifies a person or entity with image, initials, or fallback icon.

```
import { Avatar } from '@bunyan/design-system';
```

- Variants & states**
- Four sizes; image, initials, fallback, image-error fallback.
- Accessibility**
- Exposes an image role and meaningful label.
- Example**
- `<Avatar name="Fatimah Ali" source={{ uri }} />.`
- Do/don't**
- Provide `name`; do not use an avatar as the only identity label in critical flows.
- Edge cases**
- Initials use the first two whitespace-separated words.

AvatarProps

PROP	TYPE	DEFAULT
<code>source</code>	<code>ImageSourcePropType</code>	—
<code>name</code>	<code>string</code>	—
<code>size</code>	<code>'small' 'medium' 'large' 'xlarge'</code>	<code>'medium'</code>
<code>accessibilityLabel</code>	<code>string</code>	—

size="small"


image


initials


fallback icon

size="medium"


image


initials


fallback icon

size="large"


image


initials


fallback icon

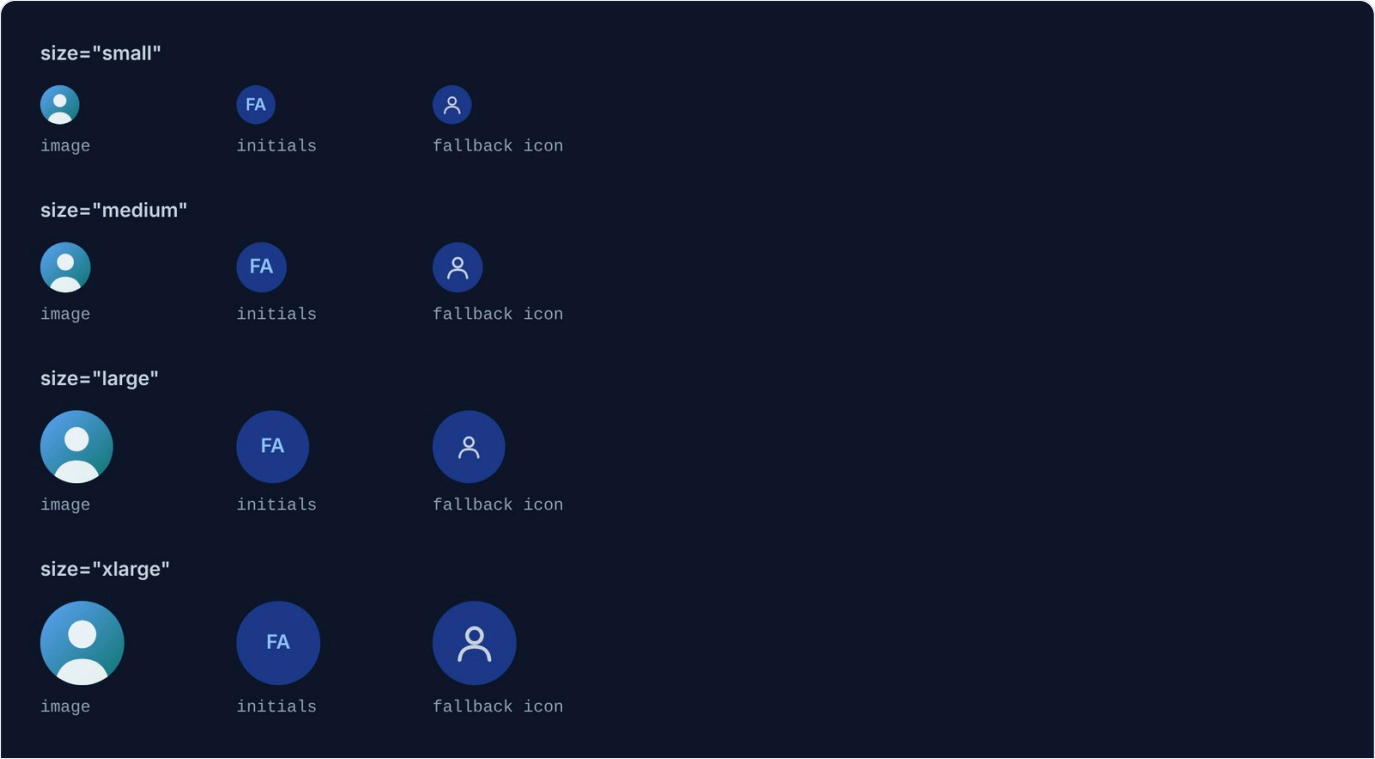
size="xlarge"


image

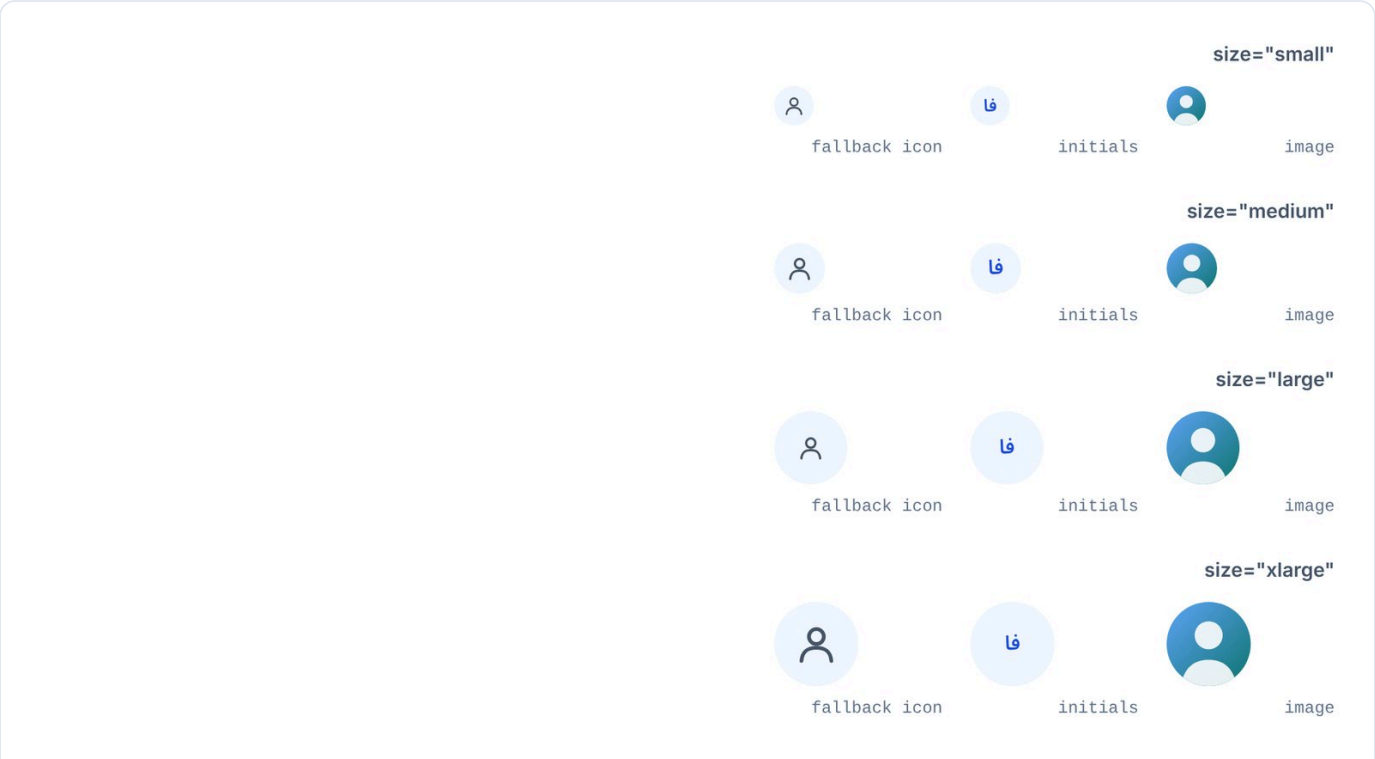

initials


fallback icon

All variants — light theme



Dark theme



Arabic (ar-SA) · RTL

20 Divider

Separates adjacent content groups.

```
import { Divider } from '@bunyan/design-system';
```

- Variants & states**
- Horizontal/vertical and four inset levels.
- Accessibility**
- Decorative by default (hidden from screen readers); `decorative={false}` exposes the separator role.
- Example**
- `<Divider inset="medium" />`.
- Do/don't**
- Use spacing before adding dividers; do not over-segment simple layouts.
- Edge cases**
- Vertical dividers need a parent with defined height.

DividerProps

PROP	TYPE	DEFAULT
<code>orientation</code>	<code>'horizontal' 'vertical'</code>	<code>'horizontal'</code>
<code>inset</code>	<code>'none' 'small' 'medium' 'large'</code>	<code>'none'</code>
<code>decorative</code>	<code>boolean</code>	<code>true</code>

orientation="horizontal" inset="none"

orientation="horizontal" inset="small"

orientation="horizontal" inset="medium"

orientation="horizontal" inset="large"

orientation="vertical"

Left | Right

All variants — light theme

orientation="horizontal" inset="none"

orientation="horizontal" inset="small"

orientation="horizontal" inset="medium"

orientation="horizontal" inset="large"

orientation="vertical"

Left | Right

Dark theme

orientation="horizontal" inset="none"

orientation="horizontal" inset="small"

orientation="horizontal" inset="medium"

orientation="horizontal" inset="large"

orientation="vertical"

يسار | يمين

Arabic (ar-SA) · RTL

21 Card

Groups related content or exposes a single card-level action.

```
import { Card } from '@bunyan/design-system';
```

Variants & states	Elevated, outlined, filled; four padding levels; pressed.
Accessibility	Interactive cards require an accessibility label and use button semantics.
Example	<code><Card variant="elevated"><Text>Summary</Text></Card></code> .
Do/don't	Keep one information hierarchy; do not nest multiple large interactive regions.
Edge cases	Use <code>padding="none"</code> for edge-to-edge media.

CardProps

PROP	TYPE	DEFAULT
<code>children</code> required	<code>React.ReactNode</code>	—
<code>variant</code>	<code>'elevated' 'outlined' 'filled'</code>	<code>'outlined'</code>
<code>padding</code>	<code>'none' 'small' 'medium' 'large'</code>	<code>'medium'</code>
<code>onPress</code>	<code>Nullable<PressableProps['onPress']></code>	—
<code>accessibilityLabel</code>	<code>string</code> Overrides the text that's read by the screen reader when the user interacts with the element. By default, the label is constructed by traversing all the children and accumulating all the Text nodes separated by space.	—

Also accepts: `Omit<PressableProps>`, `Omit<ViewProps>` (native React Native props).

variant

Balance
SAR 12,480.00

Balance
SAR 12,480.00

Balance
SAR 12,480.00

outlined elevated filled

Open account
Tap to view

pressable (onPress)

padding

Content

Content

Content

Content

none small medium large

All variants — light theme

variant

Balance
SAR 12,480.00

Balance
SAR 12,480.00

Balance
SAR 12,480.00

outlined elevated filled

Open account
Tap to view

pressable (onPress)

padding

Content

Content

Content

Content

none small medium large

Dark theme

variant

الرصيد
SAR 12,480.00

الرصيد
SAR 12,480.00

الرصيد
SAR 12,480.00

filled elevated outlined

فتح الحساب
اضغط للعرض

pressable (onPress)

padding

محتوى

محتوى

محتوى

محتوى

large medium small none

Arabic (ar-SA) · RTL

22 ListItem

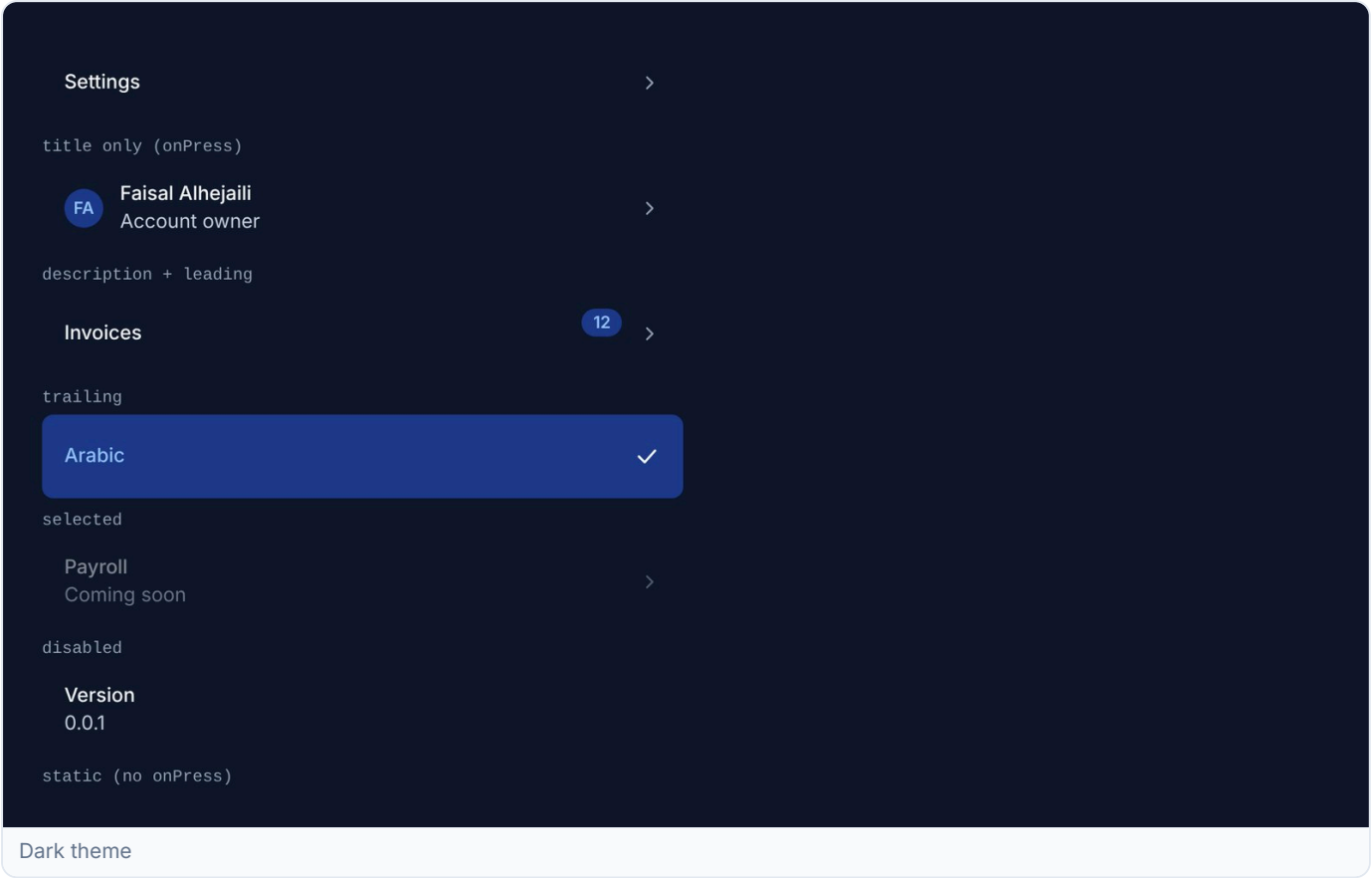
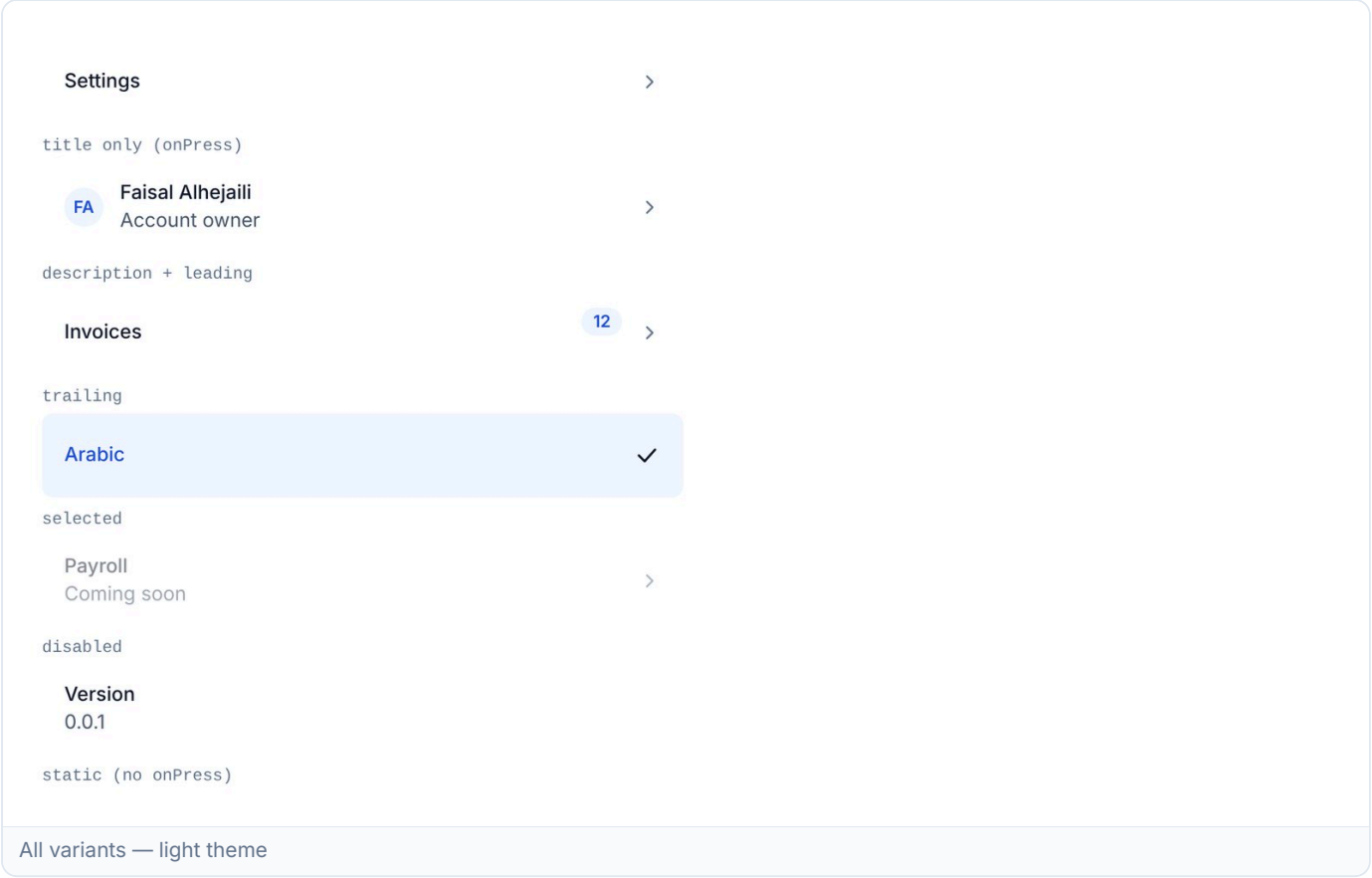
Standard row for settings, navigation, selection, or metadata.

```
import { ListItem } from '@bunyan/design-system';
```

- Variants & states**
- Static, interactive, pressed, selected, disabled.
- Accessibility**
- Interactive rows expose button role and selected state.
- Example**
- `<ListItem title="Security" description="Password and devices" onPress={openSecurity} />.`
- Do/don't**
- Put the primary label in `title`; do not hide critical actions in an unlabeled trailing slot.
- Edge cases**
- Long descriptions wrap while trailing content remains aligned.

ListItemProps

PROP	TYPE	DEFAULT
<code>title</code> required	<code>string</code>	—
<code>description</code>	<code>string</code>	—
<code>leading</code>	<code>React.ReactNode</code>	—
<code>trailing</code>	<code>React.ReactNode</code>	—
<code>onPress</code>	<code>() => void</code>	—
<code>disabled</code>	<code>boolean</code>	<code>false</code>
<code>selected</code>	<code>boolean</code>	<code>false</code>
<code>showChevron</code>	<code>boolean</code>	<code>Boolean(onPress)</code>
<code>chevronIcon</code>	<code>IconName</code>	<code>'chevron-right'</code>



<

الإعدادات

title only (onPress)

<

فيصل الحجيلي
مالك الحساب

FA

description + leading

<

12

الفواتير

trailing

✓

العربية

selected

<

الرواتب
قريبًا

disabled

الإصدار
0.0.1

static (no onPress)

Arabic (ar-SA) · RTL

23 List

Generic list/grid that renders any component once per data item — rows of `ListItem`, a wrap of `Chip`s, a grid of `Card`s, a row of `Button`s, or your own component.

```
import { List } from '@bunyan/design-system';
```

Variants & states	Column (default), row, wrapping row, horizontal scroll, n-column grid; with/without dividers; inside a <code>Card</code> ; loading skeletons; empty state.
Accessibility	Items container uses the list role and each cell the listitem role; loading is one progressbar announcement for the whole region; item accessibility comes from the rendered component.
Example	<pre><List Component={ListItem} data={users} formatItem={u => ({ title: u.name, description: u.email })} shareProps={{ onPress: open }} withDivider cardVariant="outlined" />.</pre>
Do/don't	Map API objects in <code>formatItem</code> and put common props in <code>shareProps</code> ; do not use it for thousands of rows (it renders every item — use a virtualized list for very long data).
Edge cases	Item props override <code>shareProps</code> ; <code>formatItem</code> is required by TypeScript when <code>data</code> does not match the component's props; returning <code>null</code> from <code>formatItem</code> skips the item (with <code>filterNull</code> , default on); keys default to <code>id</code> , then <code>key</code> , then index; the last grid row is padded so cells keep equal widths; spacing accepts points or token names (<code>'md'</code>); requires TypeScript ≥ 5.4 (<code>NoInfer</code>).

ListProps

PROP	TYPE	DEFAULT
<code>Component</code> required	<code>ListItemComponent<T></code> Component rendered once per item, e.g. <code>Button</code> , <code>Card</code> , <code>ListItem</code> , <code>Chip</code> , or your own.	—
<code>data</code> required	<code>readonly (D null undefined)[] null undefined</code> Items to render; one <code>Component</code> per item.	—
<code>formatItem</code>	<code>(item: D, index: number) => NoInfer<T> Partial<NoInfer<T>> null undefined</code> Map a raw item to <code>Component</code> props. Return <code>null</code> to skip it (with <code>filterNull</code>). Map a raw item to <code>Component</code> props. Required because <code>data</code> does not match <code>Component</code> 's props.	—
<code>shareProps</code>	<code>Partial<NoInfer<T>></code> Props shared by every item. Item props win on conflicts.	—
<code>keyExtractor</code>	<code>(item: D, index: number) => React.Key</code> Stable React key per raw item. Defaults to <code>id</code> , then <code>key</code> (of the raw item, then of the formatted props), then the index.	—
<code>style</code>	<code>StyleProp<ViewStyle></code> Root container style (outside the card / scroll view).	—
<code>paddingStyle</code>	<code>StyleProp<ViewStyle></code> Padding applied inside the root (inside the card when <code>cardVariant</code> is set).	—
<code>rowStyle</code>	<code>StyleProp<ViewStyle></code> Style of every row (one row per item, or per <code>columns</code> items in a grid).	—

PROP	TYPE	DEFAULT
<code>cellStyle</code>	<code>StyleProp<ViewStyle></code> Style of every cell wrapping a rendered item.	—
<code>scrollViewContentContainerStyle</code>	<code>StyleProp<ViewStyle></code> Content container style when <code>isScrolling</code> .	—
<code>flexDirection</code>	<code>'row' 'column' 'row-reverse' 'column-reverse'</code> Main axis of the list. <code>row</code> + <code>isScrolling</code> gives a horizontal carousel. Default <code>'column'</code> . RTL-aware.	<code>'column'</code>
<code>flexWrap</code>	<code>'wrap' 'nowrap' 'wrap-reverse'</code> Wrapping of items along the main axis (e.g. a cloud of Chips). Default <code>'nowrap'</code> .	<code>'nowrap'</code>
<code>columns</code>	<code>number</code> Grid columns. Items are placed in equal-width cells, <code>columns</code> per row. Default <code>1</code> .	<code>1</code>
<code>spacing</code>	<code>number keyof Theme['spacing']</code> Gap between items/rows. Default <code>'md'</code> .	—
<code>withDivider</code>	<code>boolean</code> Draw a Divider between rows.	<code>false</code>
<code>dividerSpacing</code>	<code>number keyof Theme['spacing']</code> Gap on each side of a divider. Default: same as <code>spacing</code> .	—
<code>dividerStyle</code>	<code>StyleProp<ViewStyle></code>	—
<code>cardVariant</code>	<code>'elevated' 'outlined' 'filled'</code> Wrap the whole list in a Card of this variant.	—
<code>isScrolling</code>	<code>boolean</code> Render inside a ScrollView (horizontal when <code>flexDirection</code> is a row).	<code>false</code>
<code>loading</code>	<code>boolean</code> Show Skeleton placeholders instead of data.	<code>false</code>
<code>loadingConfig</code>	<code>ListLoadingConfig</code>	—
<code>emptyForm</code>	<code>EmptyStateProps</code> EmptyState shown when there is nothing to render (after filtering) and not loading.	—
<code>filterNull</code>	<code>boolean</code> Drop <code>null</code> / <code>undefined</code> entries in <code>data</code> and <code>null</code> / <code>undefined</code> results of <code>formatItem</code> . Default <code>true</code> .	<code>true</code>
<code>ListHeaderComponent</code>	<code>ListSlot</code>	—
<code>ListFooterComponent</code>	<code>ListSlot</code>	—
<code>accessibilityLabel</code>	<code>string</code>	—
<code>testID</code>	<code>string</code>	—

ListLoadingConfig

PROP	TYPE	DEFAULT
count	number Number of placeholder items. Default 3.	—
renderItem	(index: number) => React.ReactNode Custom placeholder for one item (replaces the Skeleton).	—
accessibilityLabel	string Announced once for the whole loading region. Default 'Loading content'.	—
width	ViewStyle['width']	—
height	number	—
radius	'small' 'medium' 'large' 'pill'	—
lines	number	—

Also accepts: `Omit<SkeletonProps, 'accessibilityLabel'>` (native React Native props).

Component={ListItem} + formatItem + shareProps + withDivider + cardVariant="outlined" + header

Team members

- FA

Faisal Alhejaili

Owner

>
- SA

Sara Alqahtani

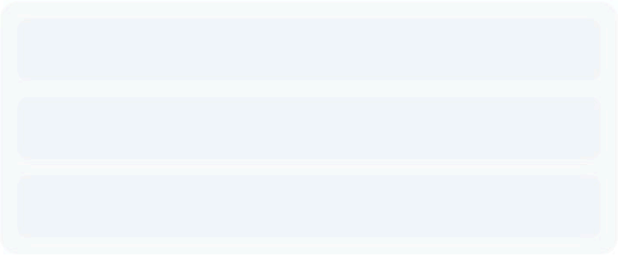
Admin

>
- OA

Omar Alharbi

Viewer

>



loading + loadingConfig={ count: 3, height: 44 }

Component={Card} columns={3} (last row padded)

Revenue
SAR 12,480

Expenses
SAR 12,480

Profit
SAR 12,480

Invoices
SAR 12,480

Component={Chip} flexDirection="row" flexWrap="wrap" spacing="sm"

- Finance
- HR
- Legal
- Sales
- Operations
- IT
- Approve
- Reject
- Details

Component={Button} flexDirection="row"

flexDirection="row" withDivider (vertical dividers) · emptyForm

Overview | Activity | Settings

flexDirection="row" withDivider



No members yet

Invite your team to get started.

data={[]} emptyForm

All variants — light theme

Component={ListItem} + formatItem + shareProps + withDivider + cardVariant="outlined" + header

Team members

- FA

Faisal Alhejaili
Owner

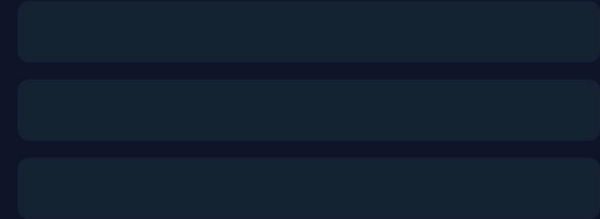
>
- SA

Sara Alqahtani
Admin

>
- OA

Omar Alharbi
Viewer

>



loading + loadingConfig={ count: 3, height: 44 }

Component={Card} columns={3} (last row padded)

Revenue

SAR 12,480

Expenses

SAR 12,480

Profit

SAR 12,480

Invoices

SAR 12,480

Component={Chip} flexDirection="row" flexWrap="wrap" spacing="sm"

- Finance
- HR
- Legal
- Sales
- Operations
- IT
- Approve
- Reject
- Details

Component={Button} flexDirection="row"

flexDirection="row" withDivider (vertical dividers) · emptyForm

Overview | Activity | Settings

flexDirection="row" withDivider

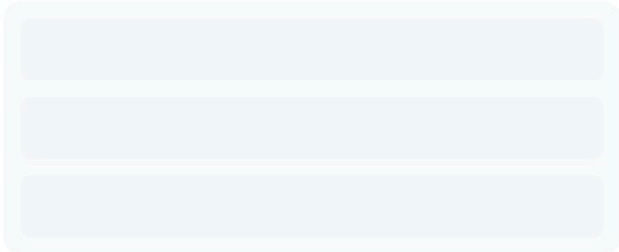
No members yet

Invite your team to get started.

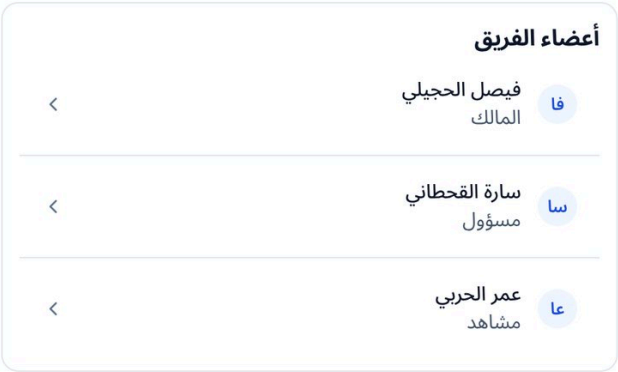
data={[]} emptyForm

Dark theme

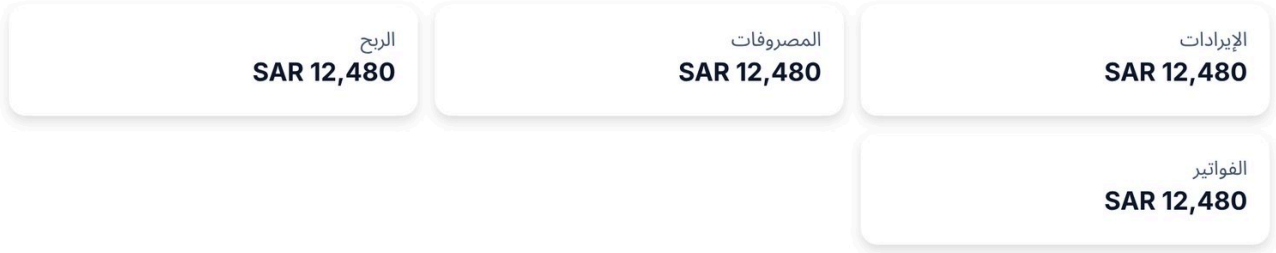
Component={ListItem} + formatItem + shareProps + withDivider + cardVariant="outlined" + header



loading + loadingConfig={ count: 3, height: 44 }



Component={Card} columns={3} (last row padded)



Component={Chip} flexDirection="row" flexWrap="wrap" spacing="sm"



Component={Button} flexDirection="row"

flexDirection="row" withDivider (vertical dividers) · emptyForm



data={[]} emptyForm

نظرة عامة | النشاط | الإعدادات
flexDirection="row" withDivider

Feedback & status

24 Alert

Persistent contextual feedback inside a screen.

```
import { Alert } from '@bunyan/design-system';
```

- Variants & states** Information, success, warning, error; actionable, dismissible.
- Accessibility** Warning/error use alert semantics; icon and color are redundant cues.
- Example** `<Alert tone="warning" title="Session expires soon" />.`
- Do/don't** Place near affected content; do not show multiple competing alerts.
- Edge cases** Long actions should move below the description naturally.

AlertProps

PROP	TYPE	DEFAULT
<code>title</code> required	<code>string</code>	—
<code>description</code>	<code>string</code>	—
<code>tone</code>	<code>'information' 'success' 'warning' 'error'</code>	<code>'information'</code>
<code>actionLabel</code>	<code>string</code>	—
<code>onAction</code>	<code>() => void</code>	—
<code>onDismiss</code>	<code>() => void</code>	—



Information message

Your changes were processed.

tone="information"



Success message

Your changes were processed.

tone="success"



Warning message

Your changes were processed.

tone="warning"



Error message

Your changes were processed.

tone="error"



Verify your email

Some features are limited.

[Resend link](#)



action + dismiss

All variants — light theme



Information message

Your changes were processed.

tone="information"



Success message

Your changes were processed.

tone="success"



Warning message

Your changes were processed.

tone="warning"



Error message

Your changes were processed.

tone="error"



Verify your email

Some features are limited.

[Resend link](#)



action + dismiss

Dark theme

عنوان التنبيه ⓘ
تمت معالجة التغييرات.

tone="information"

عنوان التنبيه ✓
تمت معالجة التغييرات.

tone="success"

عنوان التنبيه ⚠
تمت معالجة التغييرات.

tone="warning"

عنوان التنبيه ✖
تمت معالجة التغييرات.

tone="error"

×
تحقق من بريدك ⚠
بعض الميزات محدودة.
إعادة الإرسال

action + dismiss

Arabic (ar-SA) · RTL

25 Toast

Announces transient, non-blocking feedback.

```
import { ToastProvider, useToast } from '@bunyan/design-system';
```

Variants & states	Neutral, success, warning, error, information; timed or persistent.
Accessibility	Polite live region; errors use alert semantics.
Example	<code>useToast().showToast({ message: 'Saved', tone: 'success' })</code> .
Do/don't	Use concise status messages; do not use a toast for required decisions.
Edge cases	<code>duration: 0</code> persists until dismissed; the queue is capped at <code>maxVisible</code> (oldest evicted); pressing an action also dismisses; <code>dismissAll()</code> clears everything.

ToastProviderProps

PROP	TYPE	DEFAULT
<code>children</code> required	<code>React.ReactNode</code>	—
<code>maxVisible</code>	<code>number</code>	<code>3</code>
<code>defaultDuration</code>	<code>number</code>	<code>5000</code>
<code>dismissLabel</code>	<code>string</code>	<code>'Dismiss notification'</code>

ToastOptions

PROP	TYPE	DEFAULT
<code>message</code> required	<code>string</code>	—
<code>tone</code>	<code>'neutral' 'success' 'warning' 'error' 'information'</code>	—
<code>duration</code>	<code>number</code> Milliseconds before auto-dismiss. <code>0</code> keeps the toast until dismissed.	—
<code>actionLabel</code>	<code>string</code>	—
<code>onAction</code>	<code>() => void</code>	—

ToastContextValue

PROP	TYPE	DEFAULT
<code>showToast</code> required	<code>(options: ToastOptions) => number</code>	—
<code>dismissToast</code> required	<code>(id: number) => void</code>	—
<code>dismissAll</code> required	<code>() => void</code>	—

success · error with action · information

✔ Profile saved ×

⊗ Connection lost **Retry** ×

① New version available ×

Light

✔ Profile saved ×

⊗ Connection lost **Retry** ×

① New version available ×

Dark

× تم حفظ الملف ✔

× إعادة انقطع الاتصال ⊗

× يتوفر إصدار جديد ①

Arabic · RTL

26 Spinner

Indicates indeterminate activity.

```
import { Spinner } from '@bunyan/design-system';
```

- Variants & states**
- Small/large; primary/inverse.
- Accessibility**
- Progress role and text label.
- Example**
- `<Spinner label="Loading accounts" />`.
- Do/don't**
- Use a task-specific label; do not cover an entire screen for background work.
- Edge cases**
- Use Skeleton for content-shaped loading.

SpinnerProps

PROP	TYPE	DEFAULT
size	'small' 'large'	'small'
label	string	'Loading'
tone	'primary' 'inverse'	'primary'



Loading

small



Loading

large



label=""



Loading

tone=inverse

All variants — light theme



Loading

small



Loading

large



label=""



Loading

tone=inverse

Dark theme



Loading

tone=inverse



label=""



جار التحميل

large



Loading

small

Arabic (ar-SA) · RTL

27 Skeleton

Preserves layout while content is loading.

```
import { Skeleton } from '@bunyan/design-system';
```

- Variants & states**
- Single/multiple lines and four radii.
- Accessibility**
- Exposes progress semantics and respects reduced motion.
- Example**
- `<Skeleton lines={3} />.`
- Do/don't**
- Match the final layout; do not use for actions waiting on submission.
- Edge cases**
- The final line is shortened to resemble natural text.

SkeletonProps

PROP	TYPE	DEFAULT
<code>width</code>	<code>ViewStyle['width']</code>	<code>'100%'</code>
<code>height</code>	<code>number</code>	<code>—</code>
<code>radius</code>	<code>'small' 'medium' 'large' 'pill'</code>	<code>'medium'</code>
<code>lines</code>	<code>number</code>	<code>1</code>
<code>accessibilityLabel</code>	<code>string</code>	<code>'Loading content'</code>

lines=1

lines=3

radius=pill, height=40

radius=large, 64×64

All variants — light theme

lines=1

lines=3

radius=pill, height=40

radius=large, 64×64

Dark theme

radius=large, 64×64

radius=pill, height=40

lines=3

lines=1

Arabic (ar-SA) · RTL

28 EmptyState

Explains an empty collection and offers a next step.

```
import { EmptyState } from '@bunyan/design-system';
```

Variants & states	Informational and actionable.
Accessibility	Clear heading hierarchy and real button actions.
Example	<code><EmptyState title="No invoices" actionLabel="Create invoice" onAction={create} />.</code>
Do/don't	Explain why it is empty; do not imply an error when emptiness is valid.
Edge cases	Omit actions when the user cannot change the state.

EmptyStateProps

PROP	TYPE	DEFAULT
<code>title</code> required	<code>string</code>	—
<code>description</code>	<code>string</code>	—
<code>icon</code>	<code>IconName</code>	<code>'info'</code>
<code>actionLabel</code>	<code>string</code>	—
<code>onAction</code>	<code>() => void</code>	—
<code>secondaryActionLabel</code>	<code>string</code>	—
<code>onSecondaryAction</code>	<code>() => void</code>	—



No results

Try a different search term.

Clear filters

[Learn more](#)



Something went wrong

Check your connection.

Try again

ErrorState

EmptyState

All variants — light theme



No results

Try a different search term.

Clear filters

Learn more

EmptyState



Something went wrong

Check your connection.

Try again

ErrorState

Dark theme



حدث خطأ ما

تحقق من اتصالك.

حاول مجددًا

ErrorState



لا توجد نتائج

جرب كلمة بحث أخرى.

مسح الفلاتر

اعرف المزيد

EmptyState

Arabic (ar-SA) · RTL

29 ErrorState

Full-region failure with an optional retry.

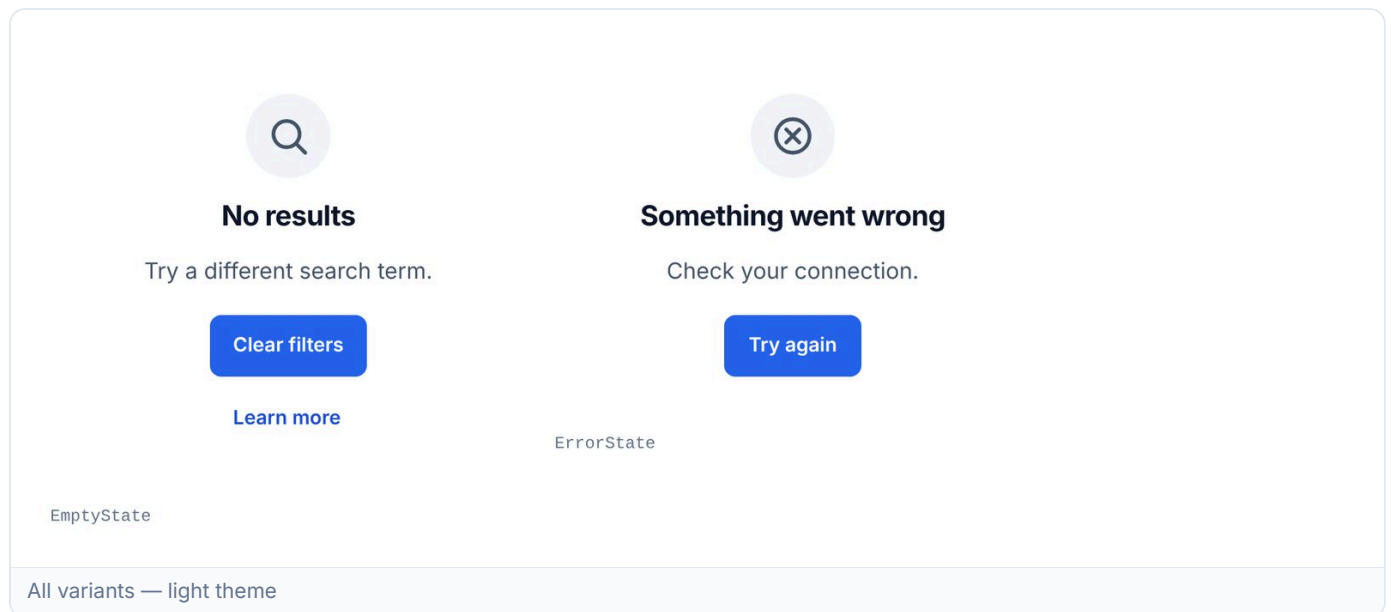
```
import { ErrorState } from '@bunyan/design-system';
```

Variants & states	Static error or retryable error.
Accessibility	Uses visible text and error icon rather than color alone.
Example	<code><ErrorState title="Unable to load" onRetry={reload} /></code> .
Do/don't	Explain recovery; do not expose raw server messages.
Edge cases	Disable or debounce retry in product code while a retry is running.

ErrorStateProps

PROP	TYPE	DEFAULT
<code>title</code> required	<code>string</code>	—
<code>description</code>	<code>string</code>	—
<code>actionLabel</code>	<code>string</code>	—
<code>onAction</code>	<code>() => void</code>	—
<code>secondaryActionLabel</code>	<code>string</code>	—
<code>onSecondaryAction</code>	<code>() => void</code>	—
<code>retryLabel</code>	<code>string</code>	<code>'Try again'</code>
<code>onRetry</code>	<code>() => void</code>	—

Also accepts: `Omit<EmptyStateProps>` (native React Native props).





No results

Try a different search term.

Clear filters

Learn more

EmptyState



Something went wrong

Check your connection.

Try again

ErrorState

Dark theme



حدث خطأ ما

تحقق من اتصالك.

حاول مجددًا

ErrorState



لا توجد نتائج

جرب كلمة بحث أخرى.

مسح الفلاتر

اعرف المزيد

EmptyState

Arabic (ar-SA) · RTL

Disclosure & navigation

30 Accordion

Progressively discloses secondary content.

```
import { Accordion } from '@bunyan/design-system';
```

- Variants & states** Collapsed, expanded, disabled.
- Accessibility** Announces expanded state and connects trigger to content.
- Example** `<Accordion title="Advanced"><AdvancedSettings /></Accordion>`.
- Do/don't** Use for optional detail; do not hide required form fields unexpectedly.
- Edge cases** Controlled mode requires the parent to update `expanded`.

AccordionProps

PROP	TYPE	DEFAULT
<code>title</code> required	<code>string</code>	—
<code>children</code> required	<code>React.ReactNode</code>	—
<code>expanded</code>	<code>boolean</code>	—
<code>defaultExpanded</code>	<code>boolean</code>	<code>false</code>
<code>onExpandedChange</code>	<code>(expanded: boolean) => void</code>	—
<code>disabled</code>	<code>boolean</code>	<code>false</code>

Account details

collapsed

Billing address

King Fahd Rd, Riyadh 12271

Security

disabled

All variants — light theme

Account details

collapsed

Billing address

King Fahd Rd, Riyadh 12271

defaultExpanded

Security

disabled

Dark theme

تفاصيل الحساب

collapsed

عنوان الفوترة

طريق الملك فهد، الرياض 12271

defaultExpanded

الأمان

disabled

Arabic (ar-SA) · RTL

31 Tabs

Switches between peer content views.

```
import { Tabs } from '@bunyan/design-system';
```

- Variants & states**
- Line/pill; selected, pressed, disabled; optional badge.
- Accessibility**
- Tablist and tab roles with selected state.
- Example**
- `<Tabs items={items} value={tab} onValueChange={setTab} />.`
- Do/don't**
- Keep labels short; do not use tabs as a stepper.
- Edge cases**
- Horizontal scrolling handles long translations; re-pressing the active tab does not fire `onValueChange`.

TabsProps

PROP	TYPE	DEFAULT
<code>items</code> required	<code>readonly TabItem[]</code>	—
<code>value</code> required	<code>string</code>	—
<code>onValueChange</code> required	<code>(value: string) => void</code>	—
<code>variant</code>	<code>'line' 'pill'</code>	<code>'line'</code>
<code>accessibilityLabel</code>	<code>string</code>	<code>'Tabs'</code>
<code>testID</code>	<code>string</code>	—

TabItem

PROP	TYPE	DEFAULT
<code>value</code> required	<code>string</code>	—
<code>label</code> required	<code>string</code>	—
<code>disabled</code>	<code>boolean</code>	—
<code>badge</code>	<code>string</code>	—
<code>accessibilityLabel</code>	<code>string</code>	—

variant="line"

Overview

Activity 12

Settings

Archive

variant="pill"

Overview

Activity 12

Settings

Archive

All variants — light theme

variant="line"

Overview

Activity 12

Settings

Archive

variant="pill"

Overview

Activity 12

Settings

Archive

Dark theme

variant="line"

الأرشيف

الإعدادات

النشاط 12

نظرة عامة

variant="pill"

الأرشيف

الإعدادات

النشاط 12

نظرة عامة

Arabic (ar-SA) · RTL

32 Tooltip

Supplies supplemental context for a focused or hovered control.

```
import { Tooltip } from '@bunyan/design-system';
```

- Variants & states**
- Top/bottom; hover, focus, long-press visibility.
- Accessibility**
- Content is also provided as an accessibility hint.
- Example**
- `<Tooltip content="Archive record"><IconButton ... /></Tooltip>.`
- Do/don't**
- Keep it brief; do not put essential instructions only in a tooltip.
- Edge cases**
- Touch users reveal tooltips with long press; the bubble stays for `touchDuration` ms after release.

TooltipProps

PROP	TYPE	DEFAULT
<code>content</code> required	<code>string</code>	—
<code>children</code> required	<code>React.ReactElement</code>	—
<code>placement</code>	<code>'top' 'bottom'</code>	<code>'top'</code>
<code>touchDuration</code>	<code>number</code>	<code>1500</code>
How long a long-press tooltip stays visible on touch devices (ms).		

Copy account number



placement="top" (keyboard focus)

Hover me

placement="bottom"

All variants — light theme

Copy account number



placement="top" (keyboard focus)

Hover me

placement="bottom"

Dark theme

مرّر هنا

placement="bottom"

نسخ رقم الحساب



placement="top" (keyboard focus)

Arabic (ar-SA) · RTL

Overlays

33 Modal

Interruptive, focused task or confirmation.

```
import { Modal } from '@bunyan/design-system';
```

- Variants & states**
- Small/medium/large; open/closed; dismissible; action loading/disabled.
- Accessibility**
- Isolates modal content and supports platform back dismissal.
- Example**
- `<Modal visible={open} onClose={close} title="Confirm">...</Modal>`.
- Do/don't**
- Keep tasks short; do not stack modals.
- Edge cases**
- Non-dismissible modals must provide a clear completion action.

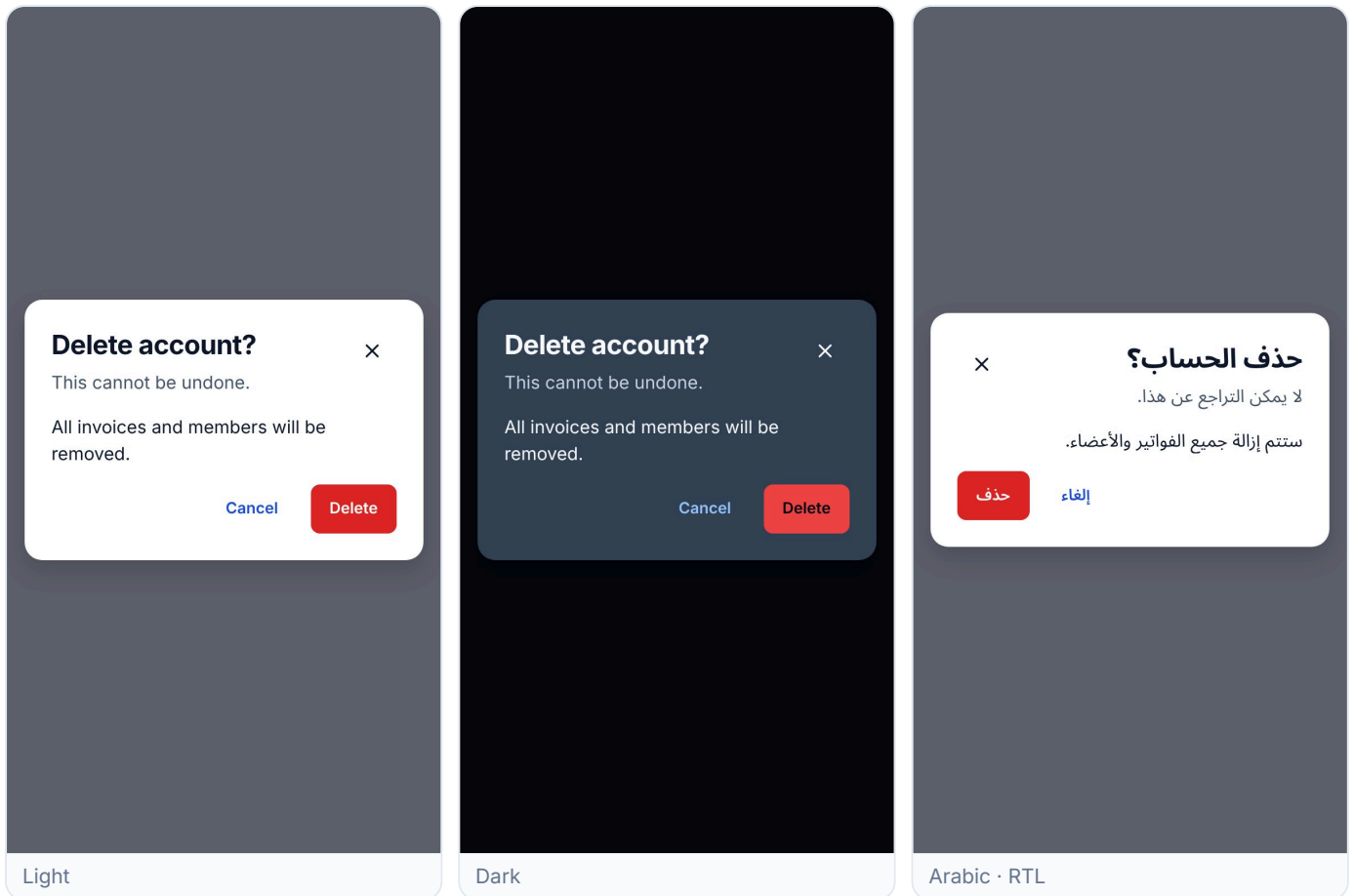
ModalProps

PROP	TYPE	DEFAULT
<code>visible</code> required	boolean	—
<code>onClose</code> required	() => void	—
<code>title</code> required	string	—
<code>description</code>	string	—
<code>children</code> required	React.ReactNode	—
<code>primaryAction</code>	ModalAction	—
<code>secondaryAction</code>	ModalAction	—
<code>dismissible</code>	boolean	true
<code>size</code>	'small' 'medium' 'large'	'medium'
<code>closeLabel</code>	string	'Close modal'
<code>testID</code>	string	—

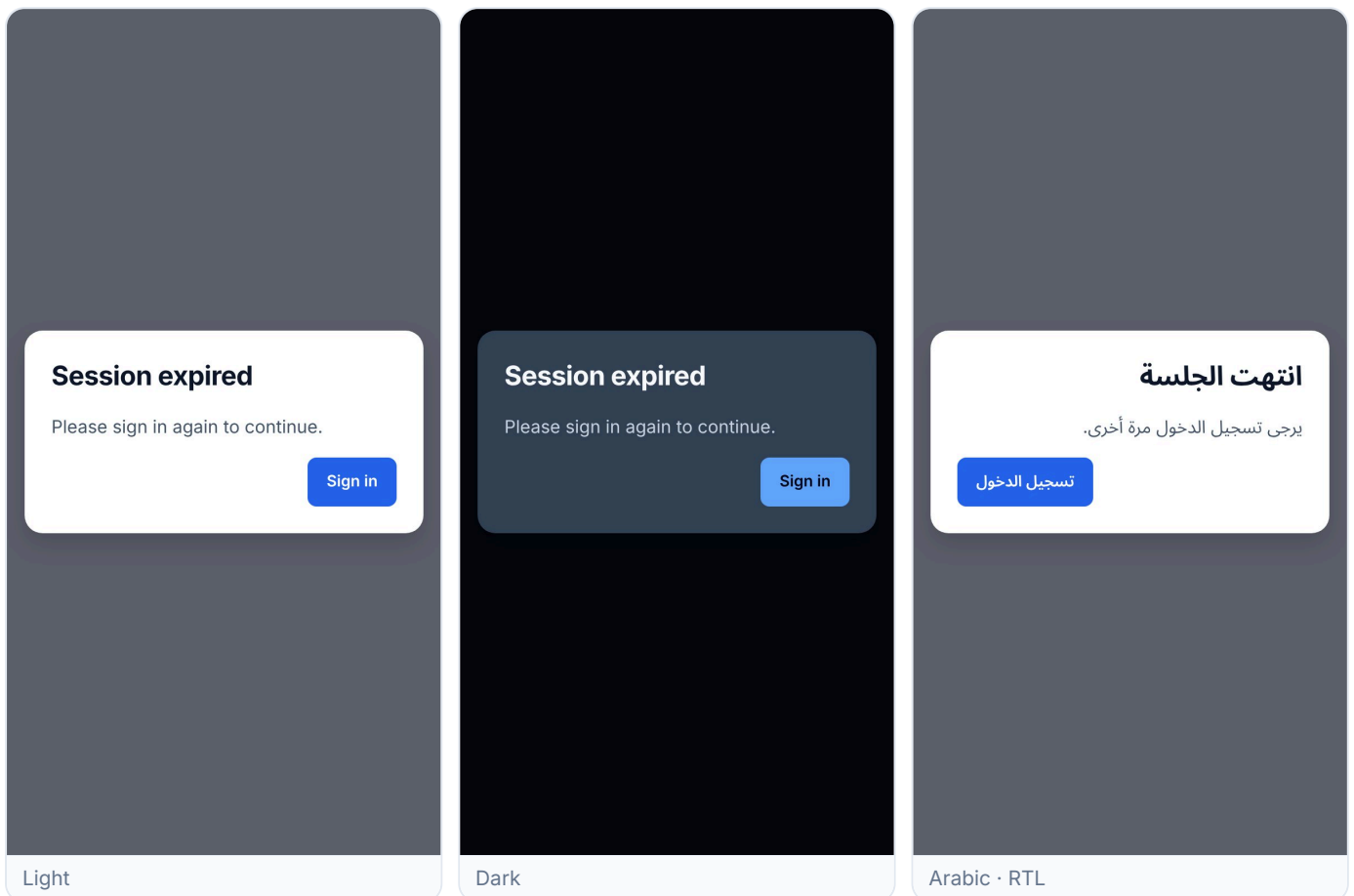
ModalAction

PROP	TYPE	DEFAULT
<code>label</code> required	string	—
<code>onPress</code> required	() => void	—
<code>variant</code>	'primary' 'secondary' 'outline' 'ghost' 'danger' 'link'	—
<code>disabled</code>	boolean	—
<code>loading</code>	boolean	—

Dismissible, with actions



dismissible={false}, size="small"



34 BottomSheet

Mobile-friendly contextual actions or compact tasks.

```
import { BottomSheet } from '@bunyan/design-system';
```

- Variants & states**
- Open/closed and dismissible/non-dismissible.
- Accessibility**
- Modal isolation and labelled close control.
- Example**
- `<BottomSheet visible={open} onClose={close} title="Actions">...</BottomSheet>`.
- Do/don't**
- Use for short choices; use Modal for complex keyboard-heavy tasks.
- Edge cases**
- Content scrolls when it exceeds 90% of viewport height.

BottomSheetProps

PROP	TYPE	DEFAULT
<code>visible</code> required	<code>boolean</code>	—
<code>onClose</code> required	<code>() => void</code>	—
<code>title</code> required	<code>string</code>	—
<code>description</code>	<code>string</code>	—
<code>children</code> required	<code>React.ReactNode</code>	—
<code>dismissible</code>	<code>boolean</code>	<code>true</code>
<code>closeLabel</code>	<code>string</code>	<code>'Close bottom sheet'</code>
<code>testID</code>	<code>string</code>	—

With list content

Share report

Choose a destination

Email>

Copy link>

Download PDF>

Light

Share report

Choose a destination

Email>

Copy link>

Download PDF>

Dark

مشاركة التقرير

اختر الوجهة

<البريد الإلكتروني

<نسخ الرابط

<تنزيل PDF

Arabic · RTL

Templates

35 NumPad

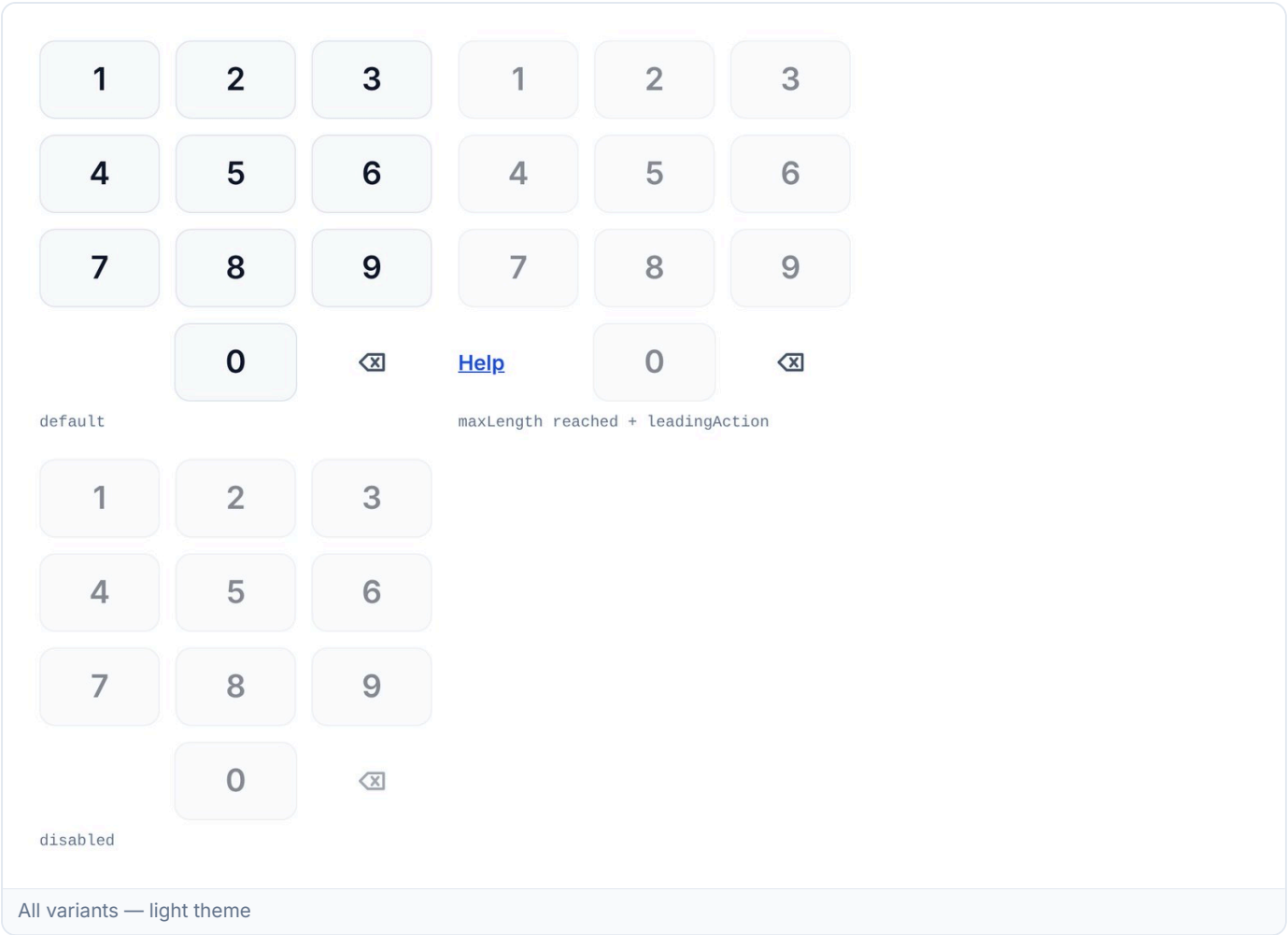
Reusable numeric entry surface for OTP, PIN, amount, and authentication flows.

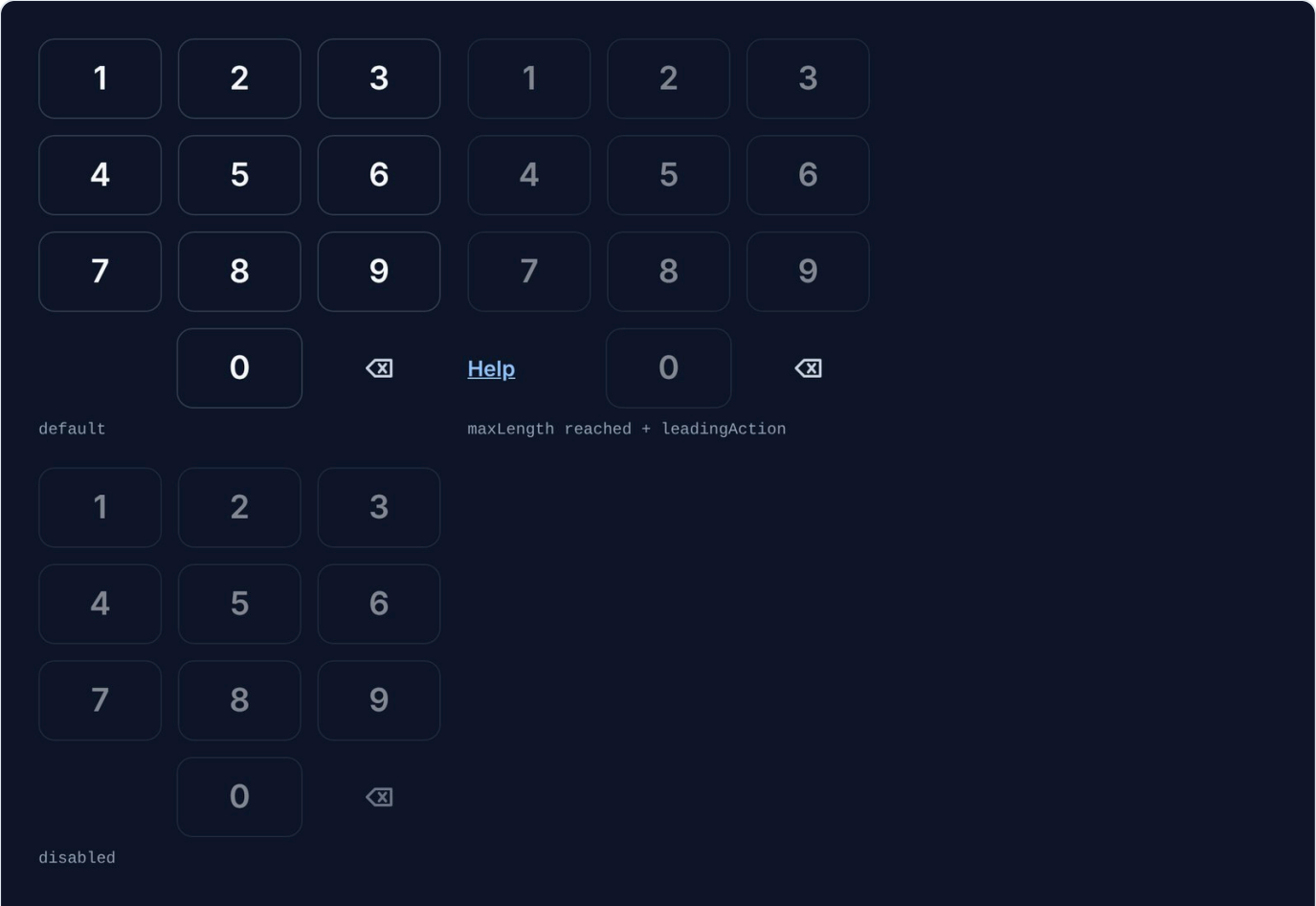
```
import { NumPad } from '@bunyan/design-system';
```

- Variants & states**
- Standard 3×4 keypad; empty, partially entered, length-limited, and disabled states.
- Accessibility**
- Every number and delete action is an individually labelled button. Numeric order stays conventional in both LTR and RTL locales.
- Example**
- `<NumPad value={pin} onChange={setPin} maxLength={4} />.`
- Do/don't**
- Keep the entered value visible elsewhere in the flow; do not use the keypad as the only accessible representation of the current value.
- Edge cases**
- `onDigitPress` and `onDelete` support reducers or secure state stores; `onChange` offers the convenient controlled-value API.

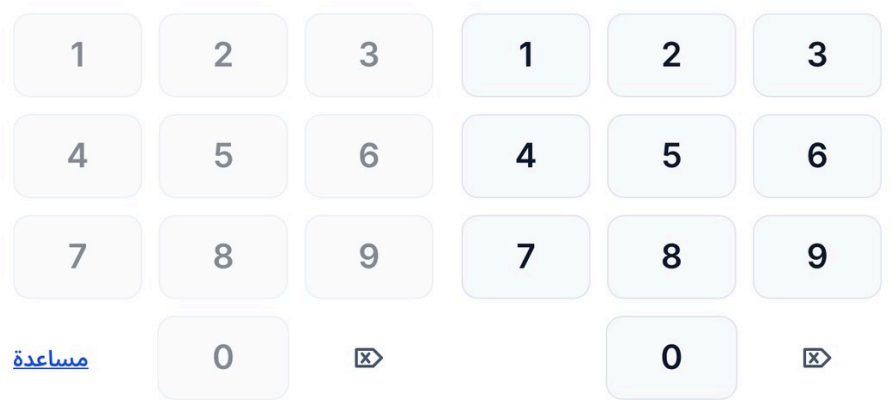
NumPadProps

PROP	TYPE	DEFAULT
<code>value</code>	<code>string</code>	<code>''</code>
<code>onChange</code>	<code>(value: string) => void</code>	—
<code>onDigitPress</code>	<code>(digit: string) => void</code>	—
<code>onDelete</code>	<code>() => void</code>	—
<code>maxLength</code>	<code>number</code>	—
<code>disabled</code>	<code>boolean</code>	<code>false</code>
<code>deleteLabel</code>	<code>string</code>	<code>'Delete last digit'</code>
<code>accessibilityLabel</code>	<code>string</code>	<code>'Numeric keypad'</code>
<code>leadingAction</code>	<code>React.ReactNode</code>	—
<code>testID</code>	<code>string</code>	—



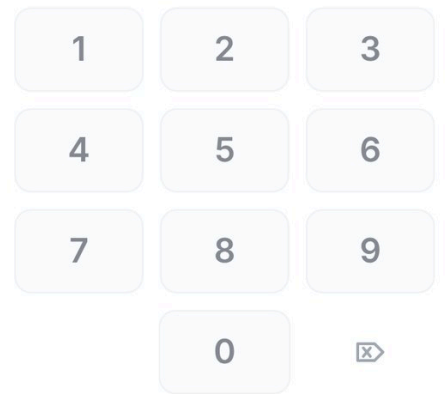


Dark theme



maxLength reached + leadingAction

default



disabled

Arabic (ar-SA) · RTL

36 OTPTemplate

Complete one-time-code verification flow using the same behavior across bottom-sheet, centered-overlay, and full-screen presentations.

```
import { OTPTemplate } from '@bunyan/design-system';
```

Variants & states	<code>bottomSheet</code> , <code>overlay</code> , <code>fullScreen</code> ; four or custom digit length; incomplete, complete, error, loading, secure, and resend-disabled states.
Accessibility	Includes a native one-time-code input for keyboard, paste, and password-manager support; slots announce entered/empty state; NumPad keys and actions are labelled.
Example	<code><OTPTemplate variant="fullScreen" value={code} onChange={setCode} onSubmit={verify} /></code> .
Do/don't	Keep verification logic outside the template; do not identify the recipient more than necessary.
Edge cases	Input is digit-filtered and length-limited; <code>autoSubmit</code> fires once per completed value; set <code>useNumPad={false}</code> when the native keyboard is preferred.

OTPTemplateProps

PROP	TYPE	DEFAULT
<code>variant</code> required	<code>'bottomSheet' 'overlay' 'fullScreen'</code>	—
<code>visible</code>	<code>boolean</code>	<code>true</code>
<code>value</code> required	<code>string</code>	—
<code>onChange</code> required	<code>(value: string) => void</code>	—
<code>onSubmit</code> required	<code>(value: string) => void</code>	—
<code>onClose</code>	<code>() => void</code>	—
<code>onResend</code>	<code>() => void</code>	—
<code>length</code>	<code>number</code>	—
<code>title</code>	<code>string</code>	<code>'Enter verification code'</code>
<code>description</code>	<code>string</code>	<code>'Enter the one-time code sent to your registered contact.'</code>
<code>destination</code>	<code>string</code>	—
<code>error</code>	<code>string</code>	—
<code>loading</code>	<code>boolean</code>	—
<code>disabled</code>	<code>boolean</code>	—
<code>secure</code>	<code>boolean</code>	—
<code>autoSubmit</code>	<code>boolean</code>	—
<code>dismissible</code>	<code>boolean</code>	<code>true</code>

PROP	TYPE	DEFAULT
<code>submitLabel</code>	string	—
<code>resendLabel</code>	string	—
<code>resendDisabled</code>	boolean	—
<code>useNumPad</code>	boolean	—
<code>testID</code>	string	—

variant= "fullScreen" + NumPad

×

Enter verification code

We sent a code to +966 5•• ••• 123

1

2

1

2

3

4

5

6

7

8

9

0

⌫

Verify

Resend code

Light

×

Enter verification code

We sent a code to +966 5•• ••• 123

1

2

1

2

3

4

5

6

7

8

9

0

⌫

Verify

Resend code

Dark

×

أدخل رمز التحقق

أرسلنا رمزاً إلى +966 5•• ••• 123

1

2

1

2

3

4

5

6

7

8

9

0

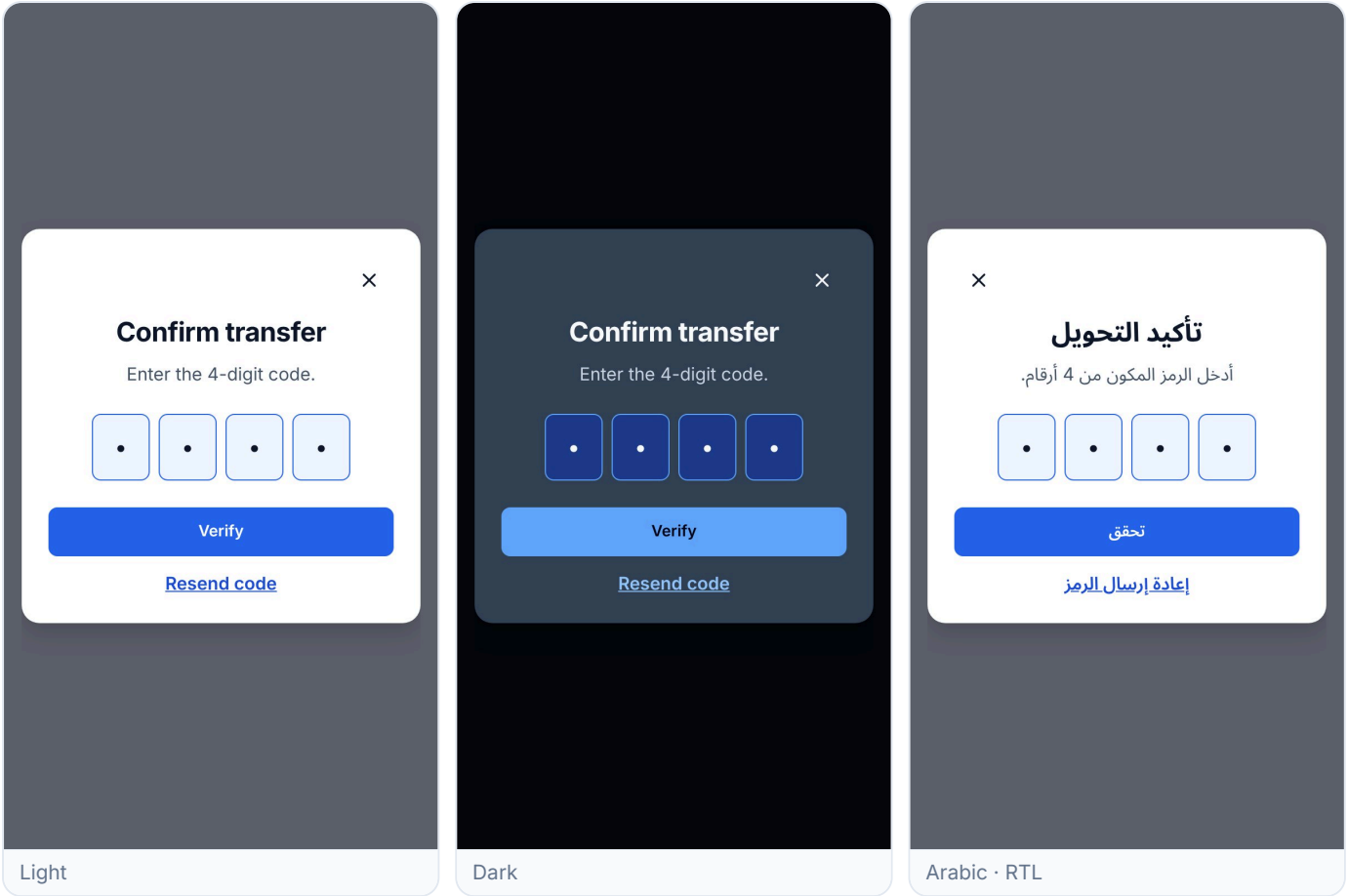
⌫

تحقق

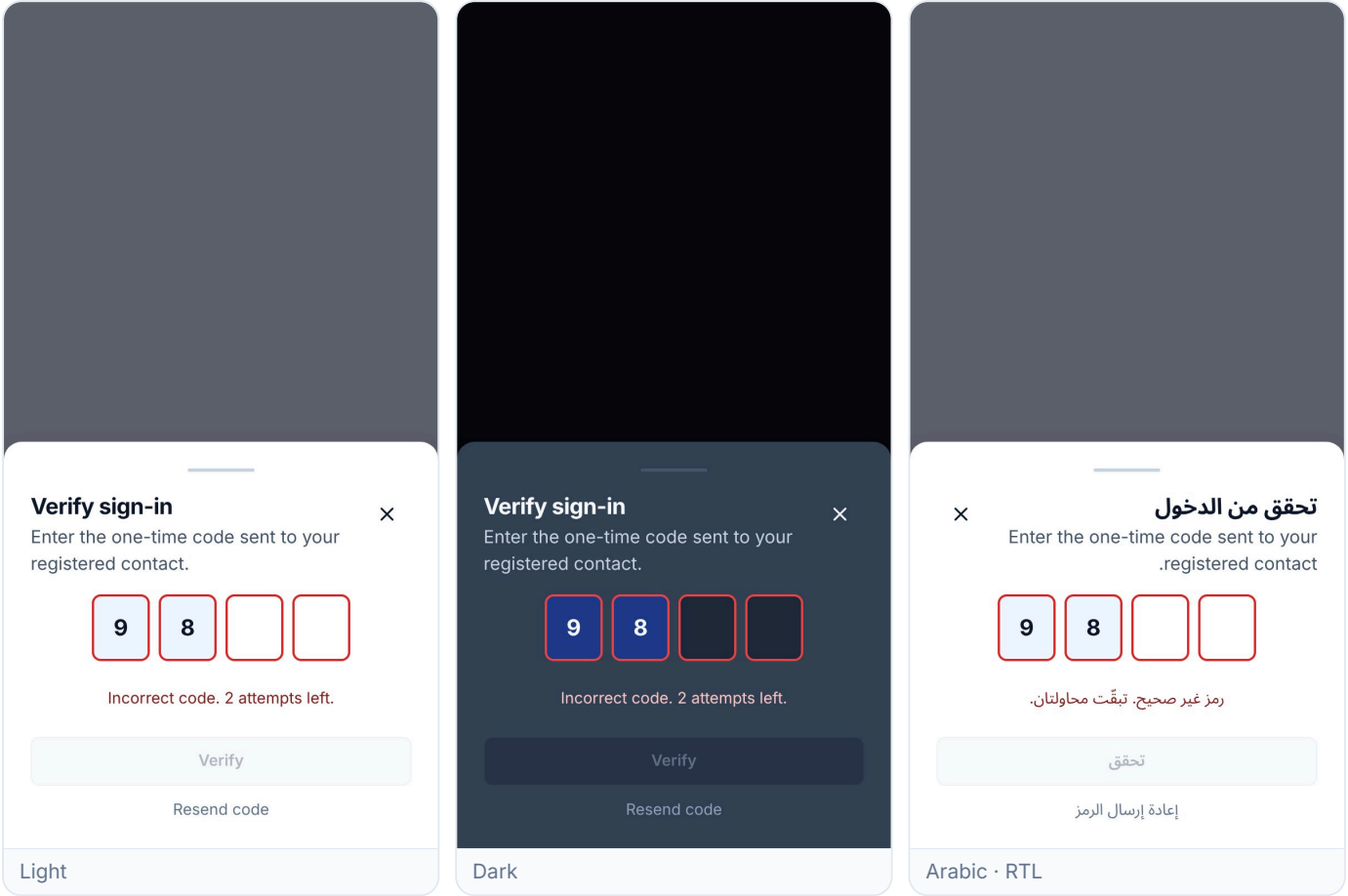
إعادة إرسال الرمز

Arabic · RTL

variant="overlay", secure, complete



variant="bottomSheet", error, resendDisabled



loading

Verifying...

Checking your code

4

8

2

1

Verify

Light

Verifying...

Checking your code

4

8

2

1

Verify

Dark

جارٍ التحقق...

جارٍ التحقق من الرمز

4

8

2

1

تحقق

Arabic · RTL

AI prompts

Paste the **context prompt** first (as a system prompt, project instructions, or the first message), then any task prompt. Replace `{placeholders}`. The same text is in [docs/AI_PROMPTS.md](#) for copy-paste.

0 Context prompt — paste first

You are working with **Bunyan** (`@bunyan/design-system`), a strongly typed React Native design system for iOS, Android and web (React 19, React Native ≥ 0.74 , TypeScript strict with `exactOptionalPropertyTypes`).

Non-negotiable rules

1. Build UI only from Bunyan exports. Import everything from `@bunyan/design-system`.
2. Never import from `react-native` in design-system code. Use `RNTheme` primitives (`RNTheme.View`, `RNTheme.Pressable`, `RNTheme.ScrollView`, `RNTheme.TextInput`, `RNTheme.RNText`, `RNTheme.Platform`, ...). Inside `src/design-system`, only `components/RNTheme/native.ts` may import `react-native`.
3. No raw colours, font sizes, spacing or radii. Read them from `useTheme().theme` (`theme.color.*`, `theme.spacing.*`, `theme.radius.*`, `theme.typography.*`), or use `themeStyle={({ theme }) => ({ ... })}` on `RNTheme` primitives.
4. Wrap the app once in `<ThemeProvider initialPreference="system" locale={locale}>` and, if toasts are used, `<ToastProvider>`.
5. Support light, dark and black themes and Arabic RTL. Use `logicalRow(direction)`, `logicalText(direction, align)` and `logicalAlignItems(...)` instead of hard-coding `row-reverse` or `left`/`right`. Number entry (OTP, NumPad) stays left-to-right.
6. Accessibility: every icon-only control has an `accessibilityLabel`; every input sits inside a `FormField` (or has a label); never use colour alone to convey state; keep touch targets ≥ 44 pt.
7. Use semantic props (`variant`, `tone`, `size`, `status`) instead of style overrides. The only public style escape hatch is `containerStyle` on `Input` (layout only).
8. All user-facing strings must be passable as props so they can be localised (labels like `clearLabel`, `confirmLabel`, `closeLabel`, `dismissLabel`, `retryLabel`).

Building blocks

- Providers/hooks: `ThemeProvider`, `useTheme()` $\rightarrow$ `{ theme, mode, preference, setPreference, direction, isRTL, locale }`; `ToastProvider`, `useToast()` $\rightarrow$ `{ showToast, dismissToast, dismissAll }`.
- Typography & icons: `Text` (`variant/tone/weight/align`), `Heading` (level 1–6), `Icon` (16 names, sizes xs–xxl, tones).
- Actions: `Button` (`primary|secondary|outline|ghost|danger|link`; `small|medium|large`; `loading`; `fullWidth`; `leading/trailing icons`; `href`/`external` to navigate), `IconButton` (`filled|outline|ghost`; required `accessibilityLabel`). `Link` is an alias of `<Button variant="link">`.
- Forms: `FormField` (`label/description/error/success/required`; `render-prop ids`), `Input`, `PasswordInput`, `SearchInput`, `TextArea`, `Checkbox`, `Radio`, `Switch`, `Select` (`options`, `searchable`), `DatePicker`.
- Display: `Badge`, `Chip`, `Avatar`, `Divider`, `Card` (`elevated|outlined|filled`; `pressable`), `ListItem`.
- Collections: `List<T, D>` renders any component per data item — `Component`, `data`, `formatItem(item, index) => props`, `shareProps` (common props; item props win), `keyExtractor`, `flexDirection`, `flexWrap`, `columns`, `spacing` (points or token), `withDivider`, `cardVariant`, `isScrolling`, `loading` + `loadingConfig`, `emptyForm` (`EmptyState` props), `ListHeaderComponent`, `ListFooterComponent`. Use it instead of `data.map(...)` in screens.
- Feedback: `Alert`, toast via `useToast`, `Spinner`, `Skeleton`, `EmptyState`, `ErrorState`.
- Disclosure/navigation: `Accordion`, `Tabs` (`line|pill`), `Tooltip`.
- Overlays: `Modal` (`primary/secondary actions`), `BottomSheet`.
- Templates: `NumPad`, `OTPTemplate` (`bottomSheet|overlay|fullScreen`).
- Theming: `RNTheme.*` themed primitives, `createThemedComponent(Component, { displayName, baseStyle, defaultProps })`.

How to answer

- Output complete, compiling TypeScript (`.tsx`) with typed props and no `any`.
- Keep state in the screen; components are controlled where possible.
- Show loading (`Skeleton`/`Spinner`), empty (`EmptyState`), and error (`ErrorState`) states for async data.
- If something is not possible with the existing components, say so and propose a new component that follows the "add a component" checklist instead of hand-rolling raw React Native.

Building app screens

B1 Build a new screen

Using the Bunyan rules above, build a `{ScreenName}` screen for a React Native app.

Purpose: {what the screen does}

Data: {shape of the data / API}

Actions: {primary action}, {secondary actions}

Requirements:

- Use only `@bunyan/design-system` components and `RNTheme` primitives; no raw colours or spacing.
- Include loading (Skeleton), empty (EmptyState) and error (ErrorState with onRetry) states.
- Must look correct in light, dark and black themes and in Arabic (`locale="ar-SA"`).
- Every interactive element has an accessible label; one primary Button per region.

Return the full `.tsx` file plus a short note on which components you used and why.

B2 Build a validated form

Create a `{FormName}` form with Bunyan.

Fields: {field: type, required?, validation rule} ...

Submit: calls `{submitFn}` and shows a success Toast; on failure show an Alert above the form.

Rules:

- Wrap every control in `FormField` and connect ids with `accessibilityLabelledBy` / `aria-describedby` via the render prop.
- Use `status="error"` on the control and `error` on FormField for messages; validate on blur and on submit.
- Use `Select` for fixed lists (searchable when > 7 options), `DatePicker` for dates, `PasswordInput` for secrets.
- The submit Button shows `loading` while submitting and is not the only way to learn about errors.

Return the component and its validation helper.

B3 Build an OTP / PIN verification flow

Implement sign-in verification using `OTPTemplate`.

Variant: {fullScreen | overlay | bottomSheet}

Code length: {4 | 6}; delivery: {SMS to +966...}

Behaviour: autoSubmit when complete; show `error` text from the API; resend is disabled for {30} seconds with a countdown in `resendLabel`; `loading` while verifying.

Use `useNumPad` on {phones without hardware keyboards}.

Keep verification logic in a hook (`useVerifyCode`) outside the template. Return the screen + hook.

B4 Render a collection with List

Render `{data description}` with Bunyan's generic `List`.

- Pick the item component: `ListItem` for rows, `Card` for tiles (`columns={2}`), `Chip` for tags (`flexDirection="row" flexWrap="wrap"`), `Button` for an action bar (`flexDirection="row"`), or a small custom component.
- Map API objects to props in `formatItem` (return `null` to skip an item) and put props common to all items (handlers, variant, size) in `shareProps`.
- Use `keyExtractor` if items have no `id`.
- Handle states with `loading` + `loadingConfig` and `emptyForm`; add `withDivider`, `cardVariant`, `ListHeaderComponent` as the design needs.
- Do not write `data.map(...)` by hand.

Return the typed component and explain the chosen layout props.

B5 Build a settings screen

Build a Settings screen with Bunyan: sections rendered with `Heading level={5}` and `ListItem` rows; toggles with `Switch`; a theme picker using `Radio` that calls `useTheme().setPreference('system' | 'light' | 'dark' | 'black')`; a language row that switches between `en` and `ar-SA`; a destructive "Delete account" row that opens a `Modal` with a `danger` primary action. Persist choices with `{storage}`. Make it fully RTL-correct.

B6 Build a dashboard / list screen

Build a `{Name}` dashboard: a `Tabs` (line) header, summary `Card`'s (elevated) with `Badge` status tones, and a list of `ListItem`'s with `Avatar` leading and trailing `Badge`. Pull-to-refresh via `RNTheme.ScrollView` `refreshControl`. Show `Skeleton lines={3}` per card while loading, `EmptyState` with a create action when empty, `ErrorState` on failure, and `useToast().showToast` after mutations.

B7 Make an existing screen theme- and RTL-ready

Here is an existing React Native screen:

```
```.tsx
{paste code}
```
```

Refactor it to Bunyan:

- Replace every `react-native` import with Bunyan components or `RNTheme` primitives.
- Replace hard-coded colours/sizes with theme tokens; replace `flexDirection: 'row'` + manual RTL code with `logicalRow(direction)`.
- Replace custom buttons/inputs with `Button`, `IconButton`, `Input`, `FormField`.
- Keep behaviour identical. List every replacement you made in a table (before → after).

B8 Choose the right feedback pattern

For each of these situations, choose between `Toast`, `Alert`, `Modal`, `BottomSheet`, `EmptyState`, `ErrorState`, `Skeleton` and `Spinner`, explain why in one sentence, and show the Bunyan code: {list of situations, e.g. "payment saved", "session about to expire", "no invoices yet", "server error loading list", "confirm delete"}

Extending & maintaining the design system

E1 Add a new component

Add a `{ComponentName}` component to Bunyan (`src/design-system/components/{ComponentName}/`).

Purpose: {purpose}

Props: {props with types, defaults}

Variants/states: {variants}

Follow the project checklist exactly:

1. `{ComponentName}.tsx` + `index.ts`; export from `components/index.ts`.
2. Import primitives only from `../RNTheme` (never `react-native`); use `useTheme()` and tokens – no raw values.
3. `memo` + `forwardRef` when it wraps a native element; ref types from `RNTheme` (`ViewRef`, `TextInputRef`).
4. RTL via `logicalRow` / `logicalText`; directional icons mirror automatically.
5. Accessibility: role, label, state (`disabled`, `selected`, `checked`, `expanded`, `busy`), ≥ 44 pt target (hitSlop for compact sizes), localisable label props.
6. Respect `exactOptionalPropertyTypes` (spread optional props conditionally).
7. Add a story to `AllComponents.stories.tsx`, a behaviour test in `tests/`, and a section in `docs/COMPONENTS.md` (Purpose, Props, Variants, Accessibility, Example, Do/Don't, Edge cases).
8. `npm run verify` must pass (typecheck + jest, including the react-native import-boundary test).

E2 Create a themed primitive with createThemedComponent

Create `{Name}` using `createThemedComponent(RNTheme.{View|Pressable|...}, { displayName: '{Name}', baseStyle: ({ theme, direction }) => ({...}), defaultProps: ({ theme }) => ({...}) })`. Base style must use tokens only. Show usage with `themeStyle` and an explicit `style` override, and add a test asserting the theme style in dark mode (`renderWithTheme(ui, { initialPreference: 'dark' })`).

E3 Add or change a design token / semantic colour

Add the semantic colour `{group}.{name}` (purpose: {purpose}).

- Add any new raw value to `tokens/primitives.ts` palette only.
- Map it in `themes/themes.ts` for light, dark and black, and extend `SemanticColors` in `themes/types.ts`.
- Check WCAG AA contrast ($\geq 4.5:1$ text, $\geq 3:1$ UI) against the surfaces it will sit on in every theme and report the ratios.
- Update `tests/tokens.test.ts` if the contract changes.

E4 Add an icon

Add the `{icon-name}` icon to `components/Icon/Icon.tsx`: extend the `IconName` union, add 24×24 stroke-only SVG children using `react-native-svg` primitives (round caps/joins, no fills), and add it to `DIRECTIONAL_ICONS` if its meaning depends on reading direction. Update the Icon story and the docs list.

E5 Write tests for a component

Write `@testing-library/react-native` tests for `{Component}` using `renderWithTheme` from `tests/test-utils`. Cover: role and accessible name, each interactive callback, disabled/loading blocking interaction, controlled vs uncontrolled behaviour, RTL (`locale: 'ar-SA'`) styles, and dark-mode rendering. Prefer behaviour assertions over snapshots. Use `act` + fake timers for anything time-based.

E6 Review a change against Bunyan rules

Review this diff as a Bunyan maintainer:

```
```diff
{paste diff}
```
```

Check and report (file:line, severity, fix): direct `react-native` imports; raw colours/spacing; missing ally roles/labels/states; RTL mistakes (hard-coded `row-reverse`, `left`/`right`); un-localisable strings; `exactOptionalPropertyTypes` violations; missing story/test/docs; behaviour regressions. Finish with a merge recommendation.

E7 Audit the design system for bugs

Audit `src/design-system` for bugs. For each component consider: stale state when props change, timers/animations/promises not cleaned up on unmount, controlled vs uncontrolled mismatches, nested pressables, double RTL mirroring, accessibility state errors, and theme values that are invisible in dark/black mode. Output a table (component, bug, reproduction, fix) and then the patches with tests.

E8 Package and release

Prepare release `{version}`: bump `package.json`, run `npm run verify`, then `npm run pack` (runs `npm i && npm pack`; `prepack` cleans and rebuilds `dist/`). Inspect the tarball file list (`tar tzf bunyan-design-system-{version}.tgz`) to confirm `dist/`, `src/`, `docs/` and `README.md` are present and no tests or stories leak into `dist/`. Write release notes grouped as Added / Fixed / Changed.

Appendix

Type aliases

| | |
|---------------------------------|--|
| <code>AlertTone</code> | <code>'information' 'success' 'warning' 'error'</code> |
| <code>BadgeTone</code> | <code>'neutral' 'primary' 'success' 'warning' 'error' 'information'</code> |
| <code>ButtonVariant</code> | <code>'primary' 'secondary' 'outline' 'ghost' 'danger' 'link'</code> |
| <code>CardVariant</code> | <code>'elevated' 'outlined' 'filled'</code> |
| <code>ComponentSize</code> | <code>'small' 'medium' 'large'</code> |
| <code>Direction</code> | <code>'ltr' 'rtl'</code> |
| <code>FeedbackStatus</code> | <code>'default' 'error' 'success'</code> |
| <code>HeadingLevel</code> | <code>1 2 3 4 5 6</code> |
| <code>IconName</code> | <code>'alert-circle' 'backspace' 'calendar' 'check' 'chevron-down' 'chevron-left' 'chevron-right' 'close' 'eye' 'eye-off' 'info' 'search' 'success' 'warning' 'error' 'user'</code> |
| <code>IconSize</code> | <code>'xs' 'sm' 'md' 'lg' 'xl' 'xxl'</code> |
| <code>IconTone</code> | <code>'primary' 'secondary' 'tertiary' 'inverse' 'success' 'warning' 'error' 'information'</code> |
| <code>ListFlexDirection</code> | <code>'row' 'column' 'row-reverse' 'column-reverse'</code> |
| <code>ListFlexWrap</code> | <code>'wrap' 'nowrap' 'wrap-reverse'</code> |
| <code>ListItemComponent</code> | <code>export type ListItemComponent<T> = React.ComponentType<T> ((props: T) => React.ReactNode)</code> |
| <code>ListSlot</code> | <code>React.ReactNode React.ComponentType</code> |
| <code>ListSpacing</code> | <code>number keyof Theme['spacing']</code> |
| <code>LogicalAlign</code> | <code>'start' 'center' 'end'</code> |
| <code>OTPTemplateVariant</code> | <code>'bottomSheet' 'overlay' 'fullScreen'</code> |
| <code>ScrollViewRef</code> | <code>React.ComponentRef<typeof Native.ScrollView></code> |
| <code>TextInputRef</code> | <code>React.ComponentRef<typeof Native.TextInput></code> |
| <code>TextRef</code> | <code>React.ComponentRef<typeof Native.Text></code> |
| <code>TextTone</code> | <code>'primary' 'secondary' 'tertiary' 'inverse' 'link' 'error' 'success'</code> |
| <code>TextVariant</code> | <code>'body' 'bodySmall' 'caption' 'label' 'code'</code> |
| <code>TextWeight</code> | <code>'regular' 'medium' 'semibold' 'bold'</code> |
| <code>ThemeMode</code> | <code>'light' 'dark' 'black'</code> |
| <code>ThemePreference</code> | <code>ThemeMode 'system'</code> |
| <code>ThemedComponent</code> | <code>export type ThemedComponent<C extends React.ElementType> = React.ForwardRefExoticComponent< React.PropsWithoutRef<ThemedComponentProps<C>> & React.RefAttributes<React.ComponentRef<C>> ></code> |

| | |
|-------------------|--|
| ThemedStaticStyle | export type ThemedStaticStyle<P> = P extends { style?: infer S } ? Exclude<S, (...args: never[]) => unknown> : never |
| ToastTone | 'neutral' 'success' 'warning' 'error' 'information' |
| Tokens | typeof tokens |
| ViewRef | React.ComponentRef<typeof Native.View> |

Complete runtime export list

Accordion, ActivityIndicator, Alert, Avatar, Badge, BottomSheet, Button, Card, Checkbox, Chip, DIRECTIONAL_ICONS, DatePicker, Divider, EmptyState, ErrorState, FormField, Heading, Icon, IconButton, Image, Input, KeyboardAvoidingView, Link, List, ListItem, ListRoot, ListRow, ListRowCell, Modal, NumPad, OTPTemplate, PasswordInput, Pressable, RNModal, RNSwitch, RNText, Radio, ScrollView, SearchInput, Select, Skeleton, Spinner, Switch, Tabs, Text, TextArea, TextInput, ThemeContext, ThemeProvider, ToastProvider, Tooltip, View, blackTheme, borderWidth, breakpoint, componentHeight, createThemedComponent, darkTheme, fontFamilyFor, heightForSize, horizontalPaddingForSize, iconSize, iconSizeForComponent, iconTokenForSize, isRTLLocale, lightTheme, logicalAlignItems, logicalFlexDirection, logicalRow, logicalText, logicalTextAlign, mergeIds, mergeThemedStyles, motion, nativeDirection, needsMirroring, opacity, palette, radius, shadow, spacing, statusBorderColor, themes, tokens, typography, useTheme, useToast, zIndex

Known limitations

- The `DatePicker` uses the native community picker; on web, supply a web implementation or replace the adapter.
- `Tooltip` wraps its child in a focusable `Pressable`; when the child is itself a button, keyboard users meet two tab stops.
- `BottomSheet` and `Modal` do not yet add bottom safe-area padding; wrap content in your safe-area view on devices with a home indicator.
- Screenshots were rendered with `react-native-web`; native platforms render system switches, spinners and pickers with their own look.